

Tema 7. Lenguaje de definición de datos

1. Introducción
2. Manipulación de bases de datos
3. Creación de tablas
4. Tipos de datos
5. Consultar las tablas de la base de datos
6. Borrar tablas
7. Modificar tablas
8. Restricciones
9. Vistas
10. Secuencias
11. Índices
12. Optimización de consultas e índices



1. Introducción



1. Introducción

¿Qué es
el DDL?

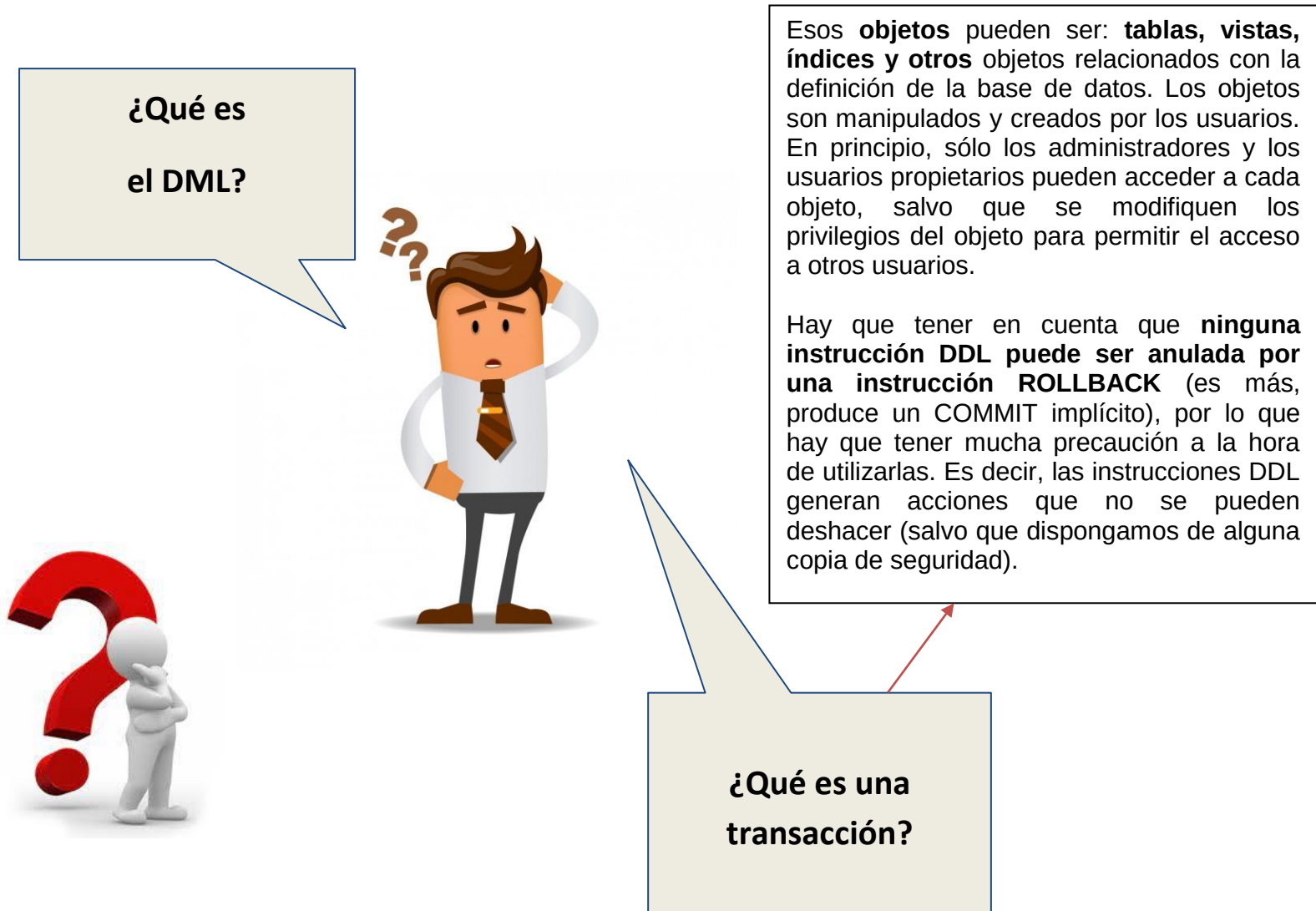
El **Lenguaje de Definición de Datos** o **DDL** (por sus siglas en inglés) es la parte del lenguaje SQL que realiza la función de definición de datos del SGBD.

Fundamentalmente se encarga de la **creación, modificación y eliminación de los objetos de la base de datos** (es decir, manejo de los metadatos). Por supuesto, es el encargado de la creación de las tablas.



¿Con qué objetos
trabaja?

1. Introducción



¿Qué es el DML?

Esos **objetos** pueden ser: **tablas, vistas, índices y otros** objetos relacionados con la definición de la base de datos. Los objetos son manipulados y creados por los usuarios. En principio, sólo los administradores y los usuarios propietarios pueden acceder a cada objeto, salvo que se modifiquen los privilegios del objeto para permitir el acceso a otros usuarios.

Hay que tener en cuenta que **ninguna instrucción DDL puede ser anulada por una instrucción ROLLBACK** (es más, produce un COMMIT implícito), por lo que hay que tener mucha precaución a la hora de utilizarlas. Es decir, las instrucciones DDL generan acciones que no se pueden deshacer (salvo que dispongamos de alguna copia de seguridad).

¿Qué es una transacción?

The illustration features a man in a white shirt and tie looking confused with question marks above his head. To his left, a 3D white figure stands next to a large red question mark. Two callout boxes are present: one on the left asking '¿Qué es el DML?' and one on the right asking '¿Qué es una transacción?'. A red arrow points from the right box to the text area above it.

2. Manipulación de datos

CREAR UNA BASE DE DATOS

La sentencia **CREATE DATABASE (ó CREATE SCHEMA)** permite crear una nueva base de datos. Lógicamente, para poder crear una base de datos, es necesario tener el privilegio global **CREATE**. Sintaxis utilizada:

```
CREATE [OR REPLACE] {DATABASE | SCHEMA} [IF NOT EXISTS] db_name
    [create_specification] ...

create_specification:
    [DEFAULT] CHARACTER SET [=] charset_name
    | [DEFAULT] COLLATE [=] collation_name
    | COMMENT [=] 'comment'
```

- **IF NOT EXISTS** crea la base de datos **sólo si no existe una base de datos con el mismo nombre**.
- **CHARACTER SET (conjunto de caracteres)** define **cómo y qué caracteres serán almacenados** para soportar un particular lenguaje o lenguajes. Si no se especifica conjunto de caracteres en la creación de la base de datos, se usará por defecto *latin1*. Para ver cuáles son los conjuntos de caracteres disponibles se puede usar la sentencia *SHOW CHARACTER SET*;
- **COLLATE (cotejamiento)** define el **orden utilizado cuando se comparan cadenas**, es decir, es el criterio que se va a utilizar para ordenar las cadenas de caracteres de la base de datos. Si no se especifica ninguno, se usará el que tenga asignado por defecto el set de caracteres escogido. Por ejemplo, para el set de caracteres *utf8mb4 (UTF-8 Unicode)* se usa *utf8mb4-general-ci*. Para consultar qué tipos de cotejamiento hay disponibles se puede usar la sentencia *SHOW COLLATION*; El cotejamiento puede ser: sensible a mayúsculas (*_cs* – ‘a’ y ‘A’ son diferentes), no sensible a mayúsculas (*_ci* – ‘a’ y ‘A’ son iguales) o binarios (*_bin* – dos caracteres son iguales si los valores de su representación numérica son iguales).

Tanto el conjunto de caracteres como el cotejamiento se pueden establecer a nivel de servidor, de base de datos, de tabla o de columna. Ambos trabajan “en cascada”, es decir, si en una columna no se ha especificado ninguno, se tomará el que se haya definido para su tabla; si ésta no ha definido ninguno, se tomará el definido para su base de datos y así sucesivamente.

2. Manipulación de datos

CREAR UNA BASE DE DATOS

Ejemplos:

```
CREATE DATABASE bd_tema5 CHARACTER SET utf8mb4 COLLATE 'utf8mb4_spanish_ci';
```

Unicode es un conjunto de caracteres universal, un estándar en el que se definen todos los caracteres necesarios para la escritura de la mayoría de los idiomas actuales. Los formatos de codificación que se pueden utilizar con Unicode se denominan *UTF-8* (utiliza 1 byte para representar caracteres en el conjunto ASCII, 2 bytes para caracteres de otros bloques alfabéticos europeos y del medio este y 3 bytes para caracteres de alfabetos chinos, japoneses...), *UTF-16* (utiliza 2 bytes para cualquier carácter de alfabetos “ordinarios” y 4 para los caracteres complementarios) y *UTF-32* (utiliza 4 bytes para todos los caracteres).

En MariaDB, el conjunto de caracteres *utf8mb4* es igual al *utf8*, pero permite almacenar caracteres suplementarios en 4 bytes. Esto permite, por ejemplo, que se puedan almacenar caracteres de tipo *Emoji Unicode*, que ocupan 4 bytes (<https://apps.timwhitlock.info/emoji/tables/unicode>).

Más información sobre conjunto de caracteres y cotejamiento se puede consultar en: <https://mariadb.com/kb/en/character-sets/>

ELIMINAR UNA BASE DE DATOS

La sentencia ***DROP DATABASE*** permite eliminar una base de datos. Para eliminar una base de datos, es necesario el privilegio global *DROP*.

La sentencia *DROP DATABASE* **elimina todo, sin solicitar confirmación**. Por lo tanto, hay que tener mucho cuidado con este comando.

La sintaxis es:

```
DROP {DATABASE | SCHEMA} [IF EXISTS] db_name
```

2. Manipulación de datos

MODIFICAR UNA BASE DE DATOS

La sentencia **ALTER DATABASE** permite modificar una base de datos cambiando sus características globales. Para utilizar esta sentencia es necesario el privilegio global **ALTER**. La sintaxis es:

```
ALTER {DATABASE | SCHEMA} [db_name]
    alter_specification ...
ALTER {DATABASE | SCHEMA} db_name
    UPGRADE DATA DIRECTORY NAME

alter_specification:
    [DEFAULT] CHARACTER SET [=] charset_name
    | [DEFAULT] COLLATE [=] collation_name
    | COMMENT [=] 'comment'
```

Cambiar el conjunto de caracteres o el cotejamiento de una base de datos no cambia dichas propiedades de los procedimientos o funciones almacenadas que fueron previamente creadas. Si se quiere aplicar sobre ellos las nuevas características, se necesitaría eliminarlas y volverlas a crear para aplicar los cambios de estas propiedades.

- Para **consultar el listado de bases de datos disponibles**: El comando **SHOW DATABASES;** muestra un listado con todas las bases de datos a las que tiene acceso el usuario con el que hemos conectado al SGBD.
- Para **seleccionar una base de datos**: El comando **USE nombre_base_datos;** se utiliza para indicar la base de datos con la que queremos trabajar.

3. Creación de tablas

La sentencia **CREATE TABLE** permite crear una tabla. En su forma básica, esta sentencia proporciona el nombre de la tabla seguido por una lista de sus columnas, índices y restricciones. Para crear una tabla, es necesario el privilegio CREATE para la tabla o la base de datos.

La sintaxis básica de esta sentencia es:

```
CREATE [OR REPLACE] [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name
    (create_definition,...) [table_options    ]
CREATE [OR REPLACE] [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name
    [(create_definition,...)] [table_options    ]
    select_statement
CREATE [OR REPLACE] [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name
    { LIKE old_table_name | (LIKE old_table_name) }

select_statement:
    [IGNORE | REPLACE] [AS] SELECT ...    (Some legal select statement)
```

Si se utiliza **OR REPLACE** junto al CREATE, entonces, si la tabla ya existe, en vez de devolver un error, **se eliminará la tabla existente y será reemplazada por la nueva** definición. Por lo tanto, las sentencias siguientes tendrán básicamente el mismo efecto:

```
CREATE OR REPLACE TABLE table_name (a int);
```

=

```
DROP TABLE IF EXISTS table_name;
CREATE TABLE table_name (a int);
```

Ejemplo:

CREATE TABLE proveedores (nombre VARCHAR(25));
Crea una tabla con un solo campo de tipo VARCHAR.

3. Creación de tablas

La opción **CREATE TABLE... SELECT** permite **crear una tabla conteniendo datos de otras**. Se crearán columnas en la tabla por cada campo devuelto por la consulta SELECT. La consulta sólo devuelve datos; si se quieren utilizar los mismos índices o los mismos atributos de columna en la nueva tabla, se necesitará especificarlos manualmente.

Cuando se va a copiar la estructura y datos de una tabla, con tablas muy grandes puede haber problemas de espacio en disco o de tiempos de respuesta importantes. **Se recomienda probar la consulta SELECT antes de ejecutar la creación propiamente dicha**. También sería buena prueba la de contar el número de filas de la tabla a copiar, lo que permitiría conocer el volumen de datos a transferir. Para copiar sólo la estructura de una tabla origen (y no sus datos), una buena opción es añadir, en el SELECT una condición que no se va a cumplir nunca. Por ejemplo: *CREATE TABLE Copia_Alum AS SELECT * FROM Alum WHERE 1=2;*

Ejemplo:

A partir de las tablas Emple y Depart, crear una tabla que contenga, en cada fila, el apellido del empleado y el nombre de departamento de los empleados:

CREATE TABLE ApeDepar AS SELECT apellido, dnombre FROM Emple e, Depart d WHERE e.dept_no=d.dept_no ORDER BY dnombre;

La opción **CREATE TABLE... LIKE** se utiliza para **crear una tabla vacía con la misma definición que otra**, incluyendo columnas, índices y opciones de tabla. Sin embargo, las definiciones de **claves foráneas**, así como las opciones de tabla **DATA DIRECTORY** o **INDEX DIRECTORY** especificadas en la tabla original **no serán creadas**. La sección create_definition puede tomar, entre otros, los siguientes valores:

```
create_definition:
  col_name column_definition
  | [CONSTRAINT [symbol]] PRIMARY KEY (index_col_name,...)
  | [CONSTRAINT [symbol]] FOREIGN KEY (index_col_name,...) reference_definition
  | CHECK (expr)

column_definition:
  data_type [NOT NULL | NULL] [DEFAULT default_value]
  [AUTO_INCREMENT] [UNIQUE [KEY] | [PRIMARY] KEY]

reference_definition:
  REFERENCES tbl_name (index_col_name,...)
  [ON DELETE reference_option]
  [ON UPDATE reference_option]

reference_option:
  RESTRICT | CASCADE | SET NULL | NO ACTION | SET DEFAULT

constraint_definition:
  CONSTRAINT [constraint_name] CHECK (expression)
```

- **NULL o NOT NULL:** indica si el campo permite almacenar valores nulos o no. Por defecto, NULL.
- **DEFAULT:** permite indicar un valor inicial por defecto si no especificamos ninguno en la inserción.
- **AUTO_INCREMENT:** sirve para indicar que es un campo autonumérico. Su valor se incrementa automáticamente en cada inserción de una fila. Sólo se utiliza en campos de tipo entero.
- **UNIQUE KEY:** indica que el campo es una clave secundaria, es decir, todos los valores de la columna deben ser distintos entre sí.
- **PRIMARY KEY:** para indicar que un campo o varios son clave primaria.
- **FOREIGN KEY:** para indicar relaciones entre campos de una tabla base y otra tabla hija. Las restricciones de clave externa en las que los valores de una columna hacen referencia a una clave de otra tabla, se llaman restricciones de integridad referencial.

3. Creación de tablas

A partir de MariaDB 10.2.1, se introdujo un nuevo método de introducir restricciones (CONSTRAINT):

- **CHECK (expression).**
- **CONSTRAINT [nombre_restricción] CHECK (expression).**

Se pueden deshabilitar todas las expresiones de restricción CHECK estableciendo la variable `check_constraint_checks` a OFF. Esto es útil, por ejemplo, cuando se quiere cargar una tabla que violaría algunas restricciones que posteriormente se quieran fijar.

La sección `table_option` puede tomar, entre otros, los siguientes valores:

```
table_options:
    table_option [[,] table_option] ...

table_option:
    AUTO_INCREMENT [=] value
| [DEFAULT] CHARACTER SET [=] charset_name
| [DEFAULT] COLLATE [=] collation_name
| ENGINE [=] engine_name
```

- **AUTO_INCREMENT:** Aquí podemos indicar el valor inicial que vamos a usar en el campo definido como `AUTO_INCREMENT`.
- **CHARACTER SET:** Especifica el conjunto de caracteres que vamos a utilizar en la tabla.
- **COLLATE:** Especifica el tipo de cotejamiento que vamos a utilizar en la tabla.
- **ENGINE:** Especifica el motor de almacenamiento que vamos a utilizar para la tabla. Los más habituales en MariaDB son InnoDB y MyISAM. Por defecto las tablas se crean con el motor InnoDB. El comando **SHOW ENGINES** permite ver cuáles son los motores disponibles y la variable del sistema `default_storage_engine` permite conocer o modificar el motor por defecto.

Ejemplos:

Ejemplo 1)

```
CREATE TABLE categoria (
    codigo INTEGER PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL
);

CREATE TABLE pieza (
    codigo INTEGER PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    color VARCHAR(50) NOT NULL,
    precio FLOAT NOT NULL,
    codigo_categoria INTEGER NOT NULL,
    FOREIGN KEY (codigo_categoria) REFERENCES categoria(codigo));
```

Ejemplo 2)

```
CREATE TABLE turista (
    id INTEGER AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    apellidos VARCHAR(100) NOT NULL,
    direccion VARCHAR(100) NOT NULL,
    telefono VARCHAR(9) NOT NULL);

CREATE TABLE hotel (
    id INTEGER AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    direccion VARCHAR(100) NOT NULL,
    ciudad VARCHAR(25) NOT NULL,
    plazas INTEGER NOT NULL,
    telefono VARCHAR(9) NOT NULL);
```

CREATE TABLE reserva (

```
id_turista INTEGER,
id_hotel INTEGER,
fecha_entrada DATETIME NOT NULL,
fecha_salida DATETIME NOT NULL,
regimen ENUM('MP', 'PC'),
PRIMARY KEY (id_turista, id_hotel),
FOREIGN KEY (id_turista) REFERENCES
    turista(id),
FOREIGN KEY (id_hotel) REFERENCES
    hotel(id)
);
```

Ejemplo 3)

```
CREATE TABLE categoria (
    codigo INTEGER PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

***HACER EJERCICIOS DEL 1 AL 4**

4. Tipos de datos

A la hora de crear tablas, hay que indicar el tipo de datos de cada campo. Cada SGBD ha creado tipos específicos para necesidades precisas. El tipo elegido dependerá de la precisión buscada, teniendo en cuenta el tamaño necesario para almacenar datos. Una buena decisión permite acceder rápidamente al valor. Todos los tipos de datos que maneja MariaDB se pueden consultar en su documentación oficial (<https://mariadb.com/kb/en/data-types/>).

Los más destacados son:

TIPO	DESCRIPCIÓN
Texto	
CHAR [(M)]	Cadenas de caracteres de longitud fija M (hasta 255). Por defecto 1. Cuando se almacena, la cadena se completa a la derecha con espacios hasta la longitud definida; estos espacios son automáticamente eliminados cuando la cadena se lee.
VARCHAR (M)	Cadenas de caracteres de longitud variable, máximo M (hasta 65535). Cuando se almacena, no se añade ningún espacio extra.
TINYTEXT	Columna de texto con un máximo de 255 (2^8-1) caracteres. Equivalente a VARCHAR(255).
TEXT [(M)]	Columna de texto con un máximo de 65535 ($2^{16}-1$) caracteres, pudiéndose especificar el tamaño M que se desee.
MEDIUMTEXT	Columna de texto con un máximo de 16.777.215 ($2^{24}-1$) caracteres.
LONGTEXT	Columna de texto con un máximo de 4.294.967.295 ($2^{32}-1$) caracteres.
ENUM ('valor1', 'valor2', ...)	Enumeración. Cadena que puede seleccionar un único valor, elegido de la lista de valores que se adjunta. Internamente, los valores ENUM son representado como enteros.
SET ('valor1', 'valor2', ...)	Conjunto. Cadena que puede seleccionar cero o más valores, cada uno de los cuales son escogidos de la lista de valores que se adjunta. Internamente, los valores SET son representados como enteros.
Números	
TINYINT	Entero muy pequeño. Ocupa 1 byte. Valores, de -128 a 127 (si es UNSIGNED, de 0 a 255).
SMALLINT	Entero pequeño. Ocupa 2 bytes. Valores, de -32768 a 32767 (si es UNSIGNED, de 0 a 65535).
MEDIUMINT	Entero mediano. Ocupa 3 bytes. Valores, de -8388608 a 8388607 (si es UNSIGNED, de 0 a 16777215).
INT - INTEGER	Entero normal. Ocupa 4 bytes. Valores, de -2.147.483.648 a 2.147.483.647 (si es UNSIGNED, de 0 a 4.294.967.295).
BIGINT	Entero largo. Ocupa 8 bytes.

4. Tipos de datos

DECIMAL [(M [,D])] DEC, NUMERIC y FIXED son sinónimos a DECIMAL A partir de MariaDB 10.3, NUMBER también es sinónimo a DECIMAL	Número en punto fijo (valores exactos). Utilizados cuando es necesario guardar los valores exactos sin redondeos (por ejemplo, para datos monetarios). M indica el número de dígitos en total (la precisión), con un rango de 1 a 65. D indica el número de cifras decimales, con un rango de 0 a 38 (desde MariadB 10.2.1; antes a 30). Si M se omite, por defecto se toma el valor 10. Si D se omite, por defecto se toma el valor 0. Ejemplo:																							
	<table><tr><th>Formato</th><th>Número escrito por el usuario</th><th>Se almacena como...</th></tr><tr><td>DECIMAL</td><td>7,456,123.89</td><td>7456123.89</td></tr><tr><td>DECIMAL (9)</td><td>7,456,123.89</td><td>7456124</td></tr><tr><td>DECIMAL (9,2)</td><td>7,456,123.89</td><td>7456123.89</td></tr><tr><td>DECIMAL (9,1)</td><td>7,456,123.89</td><td>7456123.9</td></tr><tr><td>DECIMAL (6)</td><td>7,456,123.89</td><td>Da error de precisión</td></tr><tr><td>DECIMAL (7,-2)</td><td>7,456,123.89</td><td>7456100</td></tr></table>	Formato	Número escrito por el usuario	Se almacena como...	DECIMAL	7,456,123.89	7456123.89	DECIMAL (9)	7,456,123.89	7456124	DECIMAL (9,2)	7,456,123.89	7456123.89	DECIMAL (9,1)	7,456,123.89	7456123.9	DECIMAL (6)	7,456,123.89	Da error de precisión	DECIMAL (7,-2)	7,456,123.89	7456100		
Formato	Número escrito por el usuario	Se almacena como...																						
DECIMAL	7,456,123.89	7456123.89																						
DECIMAL (9)	7,456,123.89	7456124																						
DECIMAL (9,2)	7,456,123.89	7456123.89																						
DECIMAL (9,1)	7,456,123.89	7456123.9																						
DECIMAL (6)	7,456,123.89	Da error de precisión																						
DECIMAL (7,-2)	7,456,123.89	7456100																						
FLOAT [(M,D)]	Número en punto flotante (valores aproximados) de 4 bytes. M indica el número de dígitos en total (precisión) y D es el número de cifras decimales. Si se omiten M y D, los valores son almacenados a los límites permitidos por el hardware.																							
DOUBLE [(M,D)] REAL es sinónimo a DOUBLE	Número en punto flotante de 8 bytes. M indica el número de dígitos en total (precisión) y D es el número de cifras decimales.																							
BIT [(M)]	Campo tipo bit. M indica el número de bits por valor, de 1 a 64. Si M se omite, por defecto es 1.																							
La mayoría de los tipos numéricos pueden utilizar los modificadores SIGNED (con signo, por defecto), UNSIGNED (sin signo) o ZEROFILL (rellena con ceros por la izquierda).																								
Booleanos																								
BOOL – BOOLEAN	Equivalente a TINYINT(1). Un valor de cero es considerado FALSE. Un valor no cero es considerado TRUE.																							

4. Tipos de datos

Fechas	
DATE	Fecha. Formato por defecto, 'YYYY-MM-DD'; otros, 'YY-MM-DD', YYMMDD, 'YYYY/MM/DD'...
TIME [(<precisión microsecs>)]	Hora. El rango es de '-838:59:59.999999' a '838:59:59.999999'. El formato en el que se muestra por defecto es 'HH:MM:SS.ssssss'; otros, 'D HH:MM:SS', 'HH:MM:SS', 'HH:MM', 'D HH:MM', 'D HH', 'SS', or 'HHMMSS'...
DATETIME	Fecha y hora en formato 'YYYY-MM-DD HH:MM:SS.fffff'. Desde MariaDB 10.3, DATE con una porción de tiempo es sinónimo del tipo DATETIME. Rango posible desde 1000-01-01 00:00:00.000000 a 9999-12-31 23:59:59.999999.
TIMESTAMP	Fecha y hora en formato 'YYYY-MM-DD HH:MM:SS.fffff'. Este campo se usa generalmente para definir en qué momento se agregó o actualizó una fila y, de manera predeterminada, se le asignará automáticamente la fecha y hora actual cuando se inserte o actualice un registro. Rango posible desde '1970-01-01 00:00:01' a '2038-01-19 03:14:07'.
YEAR [(4)]	Año en formato de 2 ó 4 dígitos (por defecto, 4). El formato de 2 dígitos se considera obsoleto. En el formato de 4 dígitos, el rango permitido es desde 1901 a 2155.
Para almacenar la fecha actual, la mayoría de los SGBD proporcionan funciones, como SYSDATE(), que devuelven ese valor.	
Datos de gran tamaño	
BLOB [(M)]	Objeto binario de un máximo de 65535 ($2^{16}-1$) bytes. Cada valor BLOB se almacena utilizando un prefijo de 2 bytes de longitud que indica el número de bytes en el valor. Si se añade el valor M, MariaDB crea una columna con un tipo BLOB lo suficientemente grande como para almacenar valores de M bytes de longitud.
MEDIUMBLOB	Objeto binario de un máximo de $2^{16}-1$ bytes. Cada valor BLOB se almacena utilizando un prefijo de 3 bytes de longitud que indica el número de bytes en el valor.
LOB	Objeto binario de un máximo de $2^{32}-1$ bytes. Cada valor BLOB se almacena utilizando un prefijo de 4 bytes de longitud que indica el número de bytes en el valor.

5. Consultar las tablas de la base de datos

El comando **DESCRIBE**, permite obtener la estructura de una tabla o una vista. El comando EXPLAIN también puede ser usado como sinónimo de DESCRIBE.

Ejemplo:

```
DESCRIBE articulos;  
-- ó  
EXPLAIN articulos;
```

Field	Type	Null	Key	Default	Extra
ARTICULO	varchar(20)	NO		NULL	
COD_FABRICANTE	smallint(6)	NO		NULL	
PESO	smallint(6)	NO		NULL	
CATEGORIA	varchar(10)	NO		NULL	
PRECIO_VENTA	decimal(6,2)	YES		NULL	
PRECIO_COSTO	decimal(6,2)	YES		NULL	
EXISTENCIAS	decimal(5,0)	YES		NULL	

Esta instrucción no es parte del SQL estándar, pero casi es considerada así ya que casi todos los SGBD la utilizan.

6. Borrar tablas

La sentencia ***DROP TABLE*** permite **eliminar una o más tablas**. Se tiene que tener el privilegio *DROP* para cada una de las tablas que se deseen eliminar. Se eliminarán tanto la estructura de la tabla como todos sus datos, así como los triggers asociados a la tabla, por lo que hay que tener cuidado con dicha sentencia.

Cuando se elimina una tabla, los privilegios del usuario de la tabla no son automáticamente eliminados.

La sintaxis básica es:

```
DROP [TEMPORARY] TABLE [IF EXISTS]
tbl_name [, tbl_name] ...
```

Se puede usar la cláusula ***IF EXISTS*** para evitar que ocurra un error si queremos borrar alguna tabla que no exista.

Si una clave foránea referencia la tabla que desea ser borrada, la tabla no se puede eliminar. En este caso, es necesario eliminar la clave foránea previamente.

La sentencia ***TRUNCATE TABLE*** vacía una tabla completamente. En términos generales, se trata de una sentencia equivalente, y más rápida, a una sentencia *DELETE* que borre todas las filas. *TRUNCATE TABLE* fallará en tablas con el motor InnoDB si existe una restricción de tipo *FOREIGN KEY* desde otras tablas a la tabla destino que se quiera trincar. También requiere el privilegio *DROP*.

La sintaxis básica es:

```
TRUNCATE [TABLE] tbl_name
```

***HACER LOS EJERCICIOS 5 Y 6**

7. Modificar tablas

En numerosas ocasiones es necesario **modificar los atributos de una tabla, añadir nuevos campos o eliminar otros**. Cuando la tabla tiene datos, hay que hacer uso de la sentencia **ALTER TABLE** para realizar este tipo de operaciones. Generalmente se necesita al menos el privilegio **ALTER** para poder ejecutar la sentencia **ALTER TABLE**. Si esta operación lleva consigo otras, como, por ejemplo, renombrar una tabla, además se necesitarán los privilegios **DROP**, **CREATE** e **INSERT**. La sintaxis es:

```
ALTER TABLE tbl_name
    [alter_specification [, alter_specification] ...]
    [partition_options]
```

alter_specification:

table_options

```
| ADD [COLUMN] col_name column_definition
    [FIRST | AFTER col_name]
| ADD [COLUMN] (col_name column_definition,...)
| ADD {INDEX|KEY} [index_name]
    [index_type] (index_col_name,...) [index_option] ...
| ADD [CONSTRAINT [symbol]] PRIMARY KEY
    [index_type] (index_col_name,...) [index_option] ...
| ADD [CONSTRAINT [symbol]]
    UNIQUE [INDEX|KEY] [index_name]
    [index_type] (index_col_name,...) [index_option] ...
| ADD FULLTEXT [INDEX|KEY] [index_name]
    (index_col_name,...) [index_option] ...
| ADD SPATIAL [INDEX|KEY] [index_name]
    (index_col_name,...) [index_option] ...
| ADD [CONSTRAINT [symbol]]
    FOREIGN KEY [index_name] (index_col_name,...)
    reference_definition
```

```
| ALGORITHM [=] {DEFAULT|INPLACE|COPY}
| ALTER [COLUMN] col_name {SET DEFAULT literal | DROP DEFAULT}
| CHANGE [COLUMN] old_col_name new_col_name column_definition
    [FIRST|AFTER col_name]
| [DEFAULT] CHARACTER SET [=] charset_name [COLLATE [=] collation_name]
| CONVERT TO CHARACTER SET charset_name [COLLATE collation_name]
| {DISABLE|ENABLE} KEYS
| {DISCARD|IMPORT} TABLESPACE
| DROP [COLUMN] col_name
| DROP {INDEX|KEY} index_name
| DROP PRIMARY KEY
| DROP FOREIGN KEY fk_symbol
```

```
| FORCE
| LOCK [=] {DEFAULT|NONE|SHARED|EXCLUSIVE}
| MODIFY [COLUMN] col_name column_definition
    [FIRST | AFTER col_name]
| ORDER BY col_name [, col_name] ...
| RENAME {INDEX|KEY} old_index_name TO new_index_name
| RENAME [TO|AS] new_tbl_name
| {WITHOUT|WITH} VALIDATION
| ADD PARTITION (partition_definition)
| DROP PARTITION partition_names
| DISCARD PARTITION {partition_names | ALL} TABLESPACE
| IMPORT PARTITION {partition_names | ALL} TABLESPACE
| TRUNCATE PARTITION {partition_names | ALL}
| COALESCE PARTITION number
| REORGANIZE PARTITION partition_names INTO (partition_definitions)
| EXCHANGE PARTITION partition_name WITH TABLE tbl_name [{WITH|WITHOUT}
    VALIDATION]
| ANALYZE PARTITION {partition_names | ALL}
| CHECK PARTITION {partition_names | ALL}
| OPTIMIZE PARTITION {partition_names | ALL}
| REBUILD PARTITION {partition_names | ALL}
| REPAIR PARTITION {partition_names | ALL}
| REMOVE PARTITIONING
| UPGRADE PARTITIONING
```

index_col_name:

```
col_name [(length)] [ASC | DESC]
```

index_type:

```
USING {BTREE | HASH}
```

index_option:

```
KEY_BLOCK_SIZE [=] value
| index_type
| WITH PARSER parser_name
| COMMENT 'string'
```

table_options:

```
table_option [[,] table_option] ...
```

7. Modificar tablas

```

table_option:
  AUTO_INCREMENT [=] value
  | AVG_ROW_LENGTH [=] value
  | [DEFAULT] CHARACTER SET [=] charset_name
  | CHECKSUM [=] {0 | 1}
  | [DEFAULT] COLLATE [=] collation_name
  | COMMENT [=] 'string'
  | COMPRESSION [=] {'ZLIB'|'LZ4'|'NONE'}
  | CONNECTION [=] 'connect_string'
  | {DATA|INDEX} DIRECTORY [=] 'absolute path to directory'
  | DELAY_KEY_WRITE [=] {0 | 1}
  | ENCRYPTION [=] {'Y' | 'N'}
  | ENGINE [=] engine_name
  | INSERT_METHOD [=] { NO | FIRST | LAST }
  | KEY_BLOCK_SIZE [=] value
  | MAX_ROWS [=] value
  | MIN_ROWS [=] value
  | PACK_KEYS [=] {0 | 1 | DEFAULT}
  | PASSWORD [=] 'string'
  | ROW_FORMAT [=] {DEFAULT|DYNAMIC|FIXED|COMPRESSED|REDUNDANT|COMPACT}
  | STATS_AUTO_RECALC [=] {DEFAULT|0|1}
  | STATS_PERSISTENT [=] {DEFAULT|0|1}
  | STATS_SAMPLE_PAGES [=] value
  | TABLESPACE tablespace_name [STORAGE {DISK|MEMORY|DEFAULT}]
  | UNION [=] (tbl_name[,tbl_name]...)

partition_options:
  (see CREATE TABLE options)

```

Desde MariaDB 10.0.2, se pueden utilizar los modificadores *IF EXISTS* y *IF NOT EXISTS* tras las siguientes opciones de las anteriores:

ADD COLUMN	[IF NOT EXISTS]
ADD INDEX	[IF NOT EXISTS]
ADD FOREIGN KEY	[IF NOT EXISTS]
ADD PARTITION	[IF NOT EXISTS]
CREATE INDEX	[IF NOT EXISTS]
DROP COLUMN	[IF EXISTS]
DROP INDEX	[IF EXISTS]
DROP FOREIGN KEY	[IF EXISTS]
DROP PARTITION	[IF EXISTS]
CHANGE COLUMN	[IF EXISTS]
MODIFY COLUMN	[IF EXISTS]
DROP INDEX	[IF EXISTS]

7. Modificar tablas

MODIFICAR UNA COLUMNA Y SUS ATRIBUTOS

Para realizar esta acción se utiliza la orden **MODIFY**. Cuando se modifica una columna, hay que especificar todos los atributos para la nueva columna. La columna ocupará el mismo lugar que la original y todos los índices a la columna se mantendrán.

En caso de modificación del tipo de datos, MariaDB intenta convertir los datos de la mejor manera posible al nuevo tipo. Si la longitud de una columna de tipo cadena se reduce, los valores almacenados en ese momento en la columna quedan truncados para adaptarse a la nueva longitud. De igual modo, si la longitud de una columna numérica disminuye, los valores demasiado grandes almacenados en ese momento en la columna son sustituidos por el valor máximo permitido por la nueva longitud. Este comportamiento se prohíbe cuando el modo SQL *strict* está activo (caso por defecto).

Ejemplos:

1)

```
CREATE TABLE t1 (a INT UNSIGNED
AUTO_INCREMENT, PRIMARY KEY((a));
ALTER TABLE t1 MODIFY a BIGINT UNSIGNED
AUTO_INCREMENT;
```

2)

```
CREATE TABLE usuario (
id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
nombre VARCHAR(25)
);
-- Se quiere modificar la columna "nombre" para que
pueda almacenar 50 caracteres y que no pueda tener
nulos.
ALTER TABLE usuario MODIFY nombre VARCHAR(50)
NOT NULL;
```

Existe otra orden, **CHANGE**, que trabaja igual que **MODIFY** excepto que también permite cambiar el nombre de la columna.

```
CREATE TABLE usuario (
id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
nombre_de_usuario VARCHAR(25)
);
```

-- Se quiere renombrar el nombre de la columna "nombre_de_usuario" como "nombre", que pueda almacenar 50 caracteres y que no pueda tener nulos.

```
ALTER TABLE usuario CHANGE nombre_de_usuario
nombre VARCHAR(50) NOT NULL;
```

La orden **ALTER** nos permite **asignar un valor por defecto a una columna o eliminar el valor por defecto** que tenga establecido.

```
CREATE TABLE usuario (
id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
nombre VARCHAR(50) NOT NULL,
sexo ENUM('H', 'M') NOT NULL
);
```

-- Se quiere que el valor por defecto de la columna sexo sea M.

```
ALTER TABLE usuario ALTER sexo SET DEFAULT 'M';
```

-- Si ahora se quisiera eliminar el valor por defecto de dicha columna, se haría:

```
ALTER TABLE usuario ALTER sexo DROP DEFAULT;
```

7. Modificar tablas

AÑADIR NUEVAS COLUMNAS

ADD permite **añadir nuevas columnas a una tabla**. Con los modificadores **FIRST** y **AFTER** podemos elegir el lugar de la tabla donde queremos insertar la nueva columna. **FIRST** coloca la nueva columna en primer lugar y **AFTER** la colocaría detrás de la columna que se especifique. Si no se especifica nada, la nueva columna se añadiría detrás de la última columna de la tabla.

Si se añade una columna **AUTO_INCREMENT** a una tabla que contiene datos, la columna se completa automáticamente con números a partir de 1.

Si se añade una columna con una cláusula **DEFAULT** a una tabla que contiene datos, la columna se completa automáticamente con el valor predeterminado.

Si se añade una columna **NOT NULL** a una tabla que contiene datos, la columna se completa automáticamente con el valor predeterminado de la columna (valor predeterminado implícito si no se ha definido ninguna cláusula **DEFAULT** explícitamente).

Ejemplos:

```
CREATE TABLE usuario (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL,
  sexo ENUM('H', 'M') NOT NULL
);
-- Se quiere añadir la columna fecha_nacimiento de tipo DATE
ALTER TABLE usuario ADD fecha_nacimiento DATE NOT NULL;
-- Se quieren añadir las columnas apellido1 y apellido2 detrás de la columna nombre.
ALTER TABLE usuario ADD apellido1 VARCHAR(50) NOT NULL AFTER nombre;
ALTER TABLE usuario ADD apellido2 VARCHAR(50) AFTER apellido1;
```

7. Modificar tablas

ELIMINAR COLUMNAS

DROP permite **eliminar una columna de la tabla**.

Si la columna forma parte de un índice, también es eliminada del índice; si es la única columna del índice, el índice es eliminado.

La sentencia *ALTER TABLE* puede contener varias cláusulas *DROP* separadas por una coma para eliminar varias columnas de una sola vez.

Ejemplos:

```
CREATE TABLE usuario (  
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(50) NOT NULL,  
  apellido1 VARCHAR(50) NOT NULL,  
  apellido2 VARCHAR(50),  
  sexo ENUM('H', 'M') NOT NULL DEFAULT 'M',  
  fecha_nacimiento DATE NOT NULL  
);  
-- Se quiere eliminar la columna fecha_nacimiento  
ALTER TABLE usuario DROP fecha_nacimiento;
```

***HACER EJERCICIO 7**

8. Restricciones

Las restricciones **son reglas de validación para los datos que entran en las tablas**, por ejemplo, admitir sólo valores positivos, admitir sólo las letras H ó M, que no se puedan repetir los valores en una columna, etc. Dicho de otra manera, una restricción es una **condición de obligado cumplimiento para una o más columnas de la tabla**.

Tal y como ya se ha comentado, los tipos de restricciones que nos vamos a encontrar son:

- Prohibir nulos: **NOT NULL**.
- Valor único: **UNIQUE**.
- Valor por defecto: **DEFAULT**.
- Clave primaria: **PRIMARY KEY**.
- Clave externa (ajena, foránea o secundaria): **FOREIGN KEY**.
- Restricciones de validación o de predicado: **CHECK**.

MariaDB soporta la implementación de **restricciones a nivel de tabla** utilizando las sentencias **CREATE TABLE** o **ALTER TABLE**. Una restricción de tabla restringe los datos que se pueden añadir a la tabla. Si se intenta insertar datos en una columna, MariaDB lanza un error.

El objetivo que se persigue con las restricciones es que los usuarios o aplicaciones que manipulen los datos tengan menos trabajo a la hora de mantener la coherencia de los datos, y sea el propio SGBD el que se encargue de ello.

Los nombres de restricción no se pueden repetir para el mismo esquema, se deben buscar nombres únicos. Es buena idea incluir de algún modo el nombre de la tabla, los campos involucrados y el tipo de restricción en el nombre de la misma. Por ejemplo, *pieza_id_uk* podría indicar que el campo *id* de la tabla *pieza* tiene una restricción de unicidad (UNIQUE).

Una recomendación posible a la hora de poner nombre a las restricciones podría ser:

- Tres letras para el nombre de la tabla.
- Carácter de subrayado.
- Tres letras con la columna afectada por la restricción.
- Carácter de subrayado.
- Dos letras con la abreviatura del tipo de restricción. La abreviatura puede ser:
 - **UK**. UNIQUE
 - **PK**. PRIMARY KEY*
 - **FK**. FOREIGN KEY
 - **CK**. CHECK (validación)

8. Restricciones

En MariaDB, si se asigna un nombre a la restricción de clave primaria, ese nombre es ignorado, avisando con un warning. Sin embargo, en otros SGBD sí es posible nombrarlas, por lo que es conveniente conocer la convención anterior.

La sintaxis para el uso de restricciones es:

```
[CONSTRAINT [symbol]] constraint_expression

constraint_expression:
| PRIMARY KEY [index_type] (index_col_name, ...) [index_option] ...
| FOREIGN KEY [index_name] (index_col_name, ...)
  REFERENCES tbl_name (index_col_name, ...)
  [ON DELETE reference_option]
  [ON UPDATE reference_option]
| UNIQUE [INDEX|KEY] [index_name]
  [index_type] (index_col_name, ...) [index_option] ...
| CHECK (check_constraints)

index_type:
  USING {BTREE | HASH | RTREE}

index_col_name:
  col_name [(length)] [ASC | DESC]

index_option:
| KEY_BLOCK_SIZE [=] value
| index_type
| WITH PARSER parser_name
| COMMENT 'string'
| CLUSTERING={YES|NO}

reference_option:
  RESTRICT | CASCADE | SET NULL | NO ACTION | SET DEFAULT
```

La sentencia ***SHOW INDEX FROM nombre_tabla;*** permite mostrar las claves (y más general, los índices) de una tabla.

8. Restricciones

PROHIBIR NULOS

La restricción **NOT NULL** permite prohibir los nulos en una determinada tabla. Esto obliga a que la columna tenga que tener obligatoriamente un valor para que sea almacenado el registro.

Se coloca durante la creación (o modificación) del campo añadiendo la palabra *NOT NULL* tras el tipo.

Ejemplos:

-- Al crear una tabla

```
CREATE TABLE cliente (dni VARCHAR(9) NOT NULL);
```

-- Al modificar una tabla (si dni se hubiera definido solo como VARCHAR(9), pero no nulo)

```
ALTER TABLE cliente MODIFY dni VARCHAR(9) NOT NULL;
```

Dependiendo del modo de funcionamiento que esté aplicado, al modificar una columna añadiendo una restricción *NOT NULL*, puede ocurrir que dé error (con un modo estricto) si la tabla ya tiene filas y tiene algún valor nulo o que no de error y tan sólo dé un warning (modo no estricto) de la situación en concreto.

Ejemplo:

```
CREATE TABLE cliente (dni VARCHAR(9), nombre VARCHAR(12));
```

```
INSERT INTO cliente VALUES ('3333333', 'Juan'), (NULL, 'María');
```

dni	nombre
3333333	Juan
NULL	María

--Si ahora se modifica la columna DNI para que no pueda almacenar nulos...

```
ALTER TABLE cliente MODIFY dni VARCHAR(9) NOT NULL;
```

-- Según el modo que está por defecto,

24 10:50:16 ALTER TABLE cliente MODIFY dni VARCHAR(9) NOT NULL 0 row(s) affected, 1 warning(s): 1265 Data truncated for column 'dni' at row 2 Records: 0 Duplicates: 0 Warnings: 1

8. Restricciones

PROHIBIR NULOS

-- La tabla pasaría a tener los siguientes datos:

dni	nombre
33333333	Juan
	María

-- Para evitar esta posible “ambigüedad”, lo mejor sería, antes de modificar la columna para que no admita nulos, utilizar una sentencia `UPDATE` para modificar los valores nulos, dándoles un valor “No Aplicable” o similar.

`UPDATE cliente SET dni='N/A' WHERE dni IS NULL;`

`ALTER TABLE cliente MODIFY dni VARCHAR(9) NOT NULL;`

dni	nombre
33333333	Juan
N/A	María

Para eliminar una restricción `NOT NULL` habría que utilizar una sentencia `ALTER TABLE MODIFY` sobre la columna en cuestión.

Ejemplo:

`ALTER TABLE cliente MODIFY dni VARCHAR(9);`

VALOR POR DEFECTO

A cada columna se le puede asignar un valor por defecto durante su creación mediante la propiedad **DEFAULT**. Se puede poner esta propiedad durante la creación o modificación de la tabla, añadiendo la palabra `DEFAULT` tras el tipo de datos del campo y colocando detrás el valor que se desea por defecto.

8. Restricciones

VALOR POR DEFECTO

Ejemplo:

```
CREATE TABLE cliente (dni VARCHAR(9) DEFAULT '00000000A', nombre VARCHAR(12));
INSERT INTO cliente SET nombre='Pedro';
```

dni	nombre
00000000A	Pedro

La palabra **DEFAULT** se puede añadir también durante la modificación de la tabla (comando `ALTER TABLE... MODIFY`).

Ejemplo:

```
ALTER TABLE cliente MODIFY dni VARCHAR(9) DEFAULT '00000000A';
```

Para eliminar una restricción de este tipo, simplemente habría que utilizar la sentencia anterior modificando la columna en cuestión sin añadir el **DEFAULT** oportuno.

CLAVE PRIMARIA

La clave primaria garantiza que no habrá nunca dos filas en la tabla con el mismo valor en la/s columna/s que compone/n la clave. La clave primaria hace que los campos que la forman sean **NOT NULL** (sin posibilidad de quedar vacíos) y que los valores de los campos sean de tipo **UNIQUE** (sin posibilidad de repetición). Sólo puede haber una clave primaria en cada tabla, aunque, como se ha dicho, puede estar formada por más de una columna. Cuando se crea una clave primaria, automáticamente se crea un índice para facilitar el acceso a ella.

8. Restricciones

CLAVE PRIMARIA

Una clave primaria construida por una sola columna puede definirse directamente en las opciones de la columna, en el CREATE TABLE, o en el ALTER TABLE. Si no, en términos generales, una clave primaria, de una o varias columnas, puede declararse por medio de una cláusula específica de definición de restricción.

Ejemplos:

1) Para un único atributo

```
CREATE TABLE cliente (dni VARCHAR(9) PRIMARY KEY, nombre VARCHAR(12));
```

-- Sería igual que hacer...

```
CREATE TABLE cliente (dni VARCHAR(9), nombre VARCHAR(12), CONSTRAINT PRIMARY KEY (dni));
```

-- E incluso...

```
CREATE TABLE cliente (dni VARCHAR(9), nombre VARCHAR(12), PRIMARY KEY (dni));
```

2) Para varios atributos es obligatorio hacerlo tras la definición de los atributos

```
CREATE TABLE cliente (dni VARCHAR(9), nombre VARCHAR(12), PRIMARY KEY (dni, nombre));
```

También se podría utilizar una sentencia **ALTER TABLE ... ADD** para añadir una clave primaria que no fue originalmente establecida en la tabla base.

Ejemplos:

1) Para un único atributo

```
CREATE TABLE
```

Ejemplo:

```
ALTER TABLE cliente ADD PRIMARY KEY (dni,nombre);
```

Para eliminar una clave primaria, se puede utilizar también la sentencia

ALTER TABLE... DROP PRIMARY KEY...

Ejemplo:

```
ALTER TABLE cliente DROP PRIMARY KEY;
```

Si se intenta insertar un elemento con un valor existente de la clave primaria, dará un error.

Ejemplo:

-- Se tiene...

```
CREATE TABLE cliente (dni VARCHAR(9) PRIMARY KEY, nombre VARCHAR(12));
```

-- Se inserta...

```
INSERT INTO cliente VALUES ('3333333', 'Pedro'), ('6666666', 'María');
```

dni	nombre
3333333	Pedro
6666666	María

-- Se desean insertar más registros...

```
INSERT INTO cliente VALUES ('44444444', 'Juan'), ('6666666', 'Luis');
```

53 16:31:11 INSERT INTO cliente VALUES ('44444444', 'Juan'), ('6666666', 'Luis') Error Code: 1062, Duplicate entry '6666666' for key 'PRIMARY'

-- No inserta ninguno de los 2 elementos, aunque sólo uno es el que muestra clave duplicada

8. Restricciones

CLAVE EXTERNA (AJENA, FORÁNEA O SECUNDARIA)

Una **clave foránea** es uno o más campos de una tabla que están relacionados con la **clave principal** (o incluso con una clave candidata) de otra tabla. Las restricciones de clave foránea también permiten controlar lo que pasa cuando una fila de la tabla primaria se elimina o la clave primaria de una fila de la tabla se modifica.

Las restricciones de clave externa sólo están implementadas actualmente en el motor InnoDB; la tabla referenciada y la tabla secundaria deben ser tablas InnoDB. Además, en MariaDB las claves foráneas no pueden ser utilizadas con *vistas*, elementos que se explicarán más adelante.

La sintaxis para establecer claves foráneas es:

```
[CONSTRAINT [symbol]] FOREIGN KEY
    [index_name] (index_col_name, ...)
    REFERENCES tbl_name (index_col_name,...)
    [ON DELETE reference_option]
    [ON UPDATE reference_option]

reference_option:
    RESTRICT | CASCADE | SET NULL | NO ACTION | SET DEFAULT
```

Symbol es el nombre dado a la restricción; no puede haber dos restricciones de clave externa con el mismo nombre en toda la base de datos. Si no se establece ningún nombre, MariaDB atribuye un nombre predeterminado a la restricción.

La cláusula **FOREIGN KEY** permite definir el nombre de las columnas que constituyen la clave externa. La cláusula **REFERENCES** permite especificar la clave referenciada mediante la restricción.

Las columnas correspondientes de la clave externa y de la clave referenciada deben ser del mismo tipo o similares; las dos claves deben ser indexadas, con las columnas en el mismo orden. Las columnas referenciadas deben ser **PRIMARY KEY** ó **UNIQUE**. El índice de la clave externa se crea automáticamente en caso de no tenerlo antes.

8. Restricciones

CLAVE EXTERNA (AJENA, FORÁNEA O SECUNDARIA)

Las cláusulas **ON DELETE** y **ON UPDATE** permiten definir lo que pasa si una clave referenciada es eliminada o modificada y hay una o varias filas que corresponden en la tabla secundaria. MariaDB propone cinco opciones:

- **RESTRICT**: impide que se puedan actualizar o eliminar las filas que tienen valores referenciados por claves foráneas. Es la opción por defecto en MariaDB.
- **CASCADE**: permite actualizar o eliminar las filas que tienen valores referenciados por claves foráneas.
- **SET NULL**: asigna el valor **NULL** a las filas que tienen valores referenciados por claves foráneas.
- **NO ACTION**: es una palabra clave del estándar SQL. En MariaDB es **equivalente a RESTRICT**.
- **SET DEFAULT**: no es posible utilizar esta opción cuando trabajamos con el motor de almacenamiento InnoDB.

También puede utilizarse este tipo de restricción en una sentencia **ALTER TABLE... ADD**.

Se puede utilizar una sentencia **ALTER TABLE ... DROP** para eliminar una restricción de clave externa.

Al usar claves foráneas hay que tener cuidado en el orden en que se crean las tablas: primero se crean las tablas base y luego las tablas de detalle (las que contienen las claves foráneas). Si no, si no está creada la tabla base, las restricciones de clave foránea darán error. Otra opción es crear las tablas sin restricciones y luego poner la clave primaria (o única) en la tabla base y añadir la restricción de clave foránea a la tabla de detalle.

Ejemplos:

```
-- Se crea la tabla base
CREATE TABLE gadget_types(
  type_id INT AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  PRIMARY KEY(type_id)
);
-- Se insertan valores en ella
INSERT INTO gadget_types(name)
VALUES ('Entertainment'),
('Computing'), ('Communication'),
('Lifestyle'), ('Cameras');
```

```
-- Se crea una segunda tabla,
relacionada con la primera
CREATE TABLE gadgets(
  gadget_id INT
  AUTO_INCREMENT,
  gadget_name VARCHAR(100)
  NOT NULL,
  type_id INT,
  PRIMARY KEY (gadget_id),
  CONSTRAINT fk_type FOREIGN
  KEY (type_id) REFERENCES
  gadget_types(type_id));
```

```
-- Se insertan valores en esta segunda tabla
INSERT INTO
gadgets(gadget_name,type_id) VALUES
('Amazon Kindle',1), ('Apple iPod',1), ('Audio
Highway Listen Up',1), ('Apple iPad',2),
('MicroSD',2), ('Apple iPhone',3),
('BlackBerry 6210',3), ('Pager',3), ('Air Taser
Model 34000',4), ('Credit Card',4),
('Zippo',4), ('Casio G-Shock DW-5000C',4),
('Nikon F',5), ('Canon EOS 5D Mark II',5);
```

```
-- Se intenta borrar una fila de la tabla
" gadget_types "
DELETE FROM gadget_types WHERE type_id = 1;
```

9 11:01:43 DELETE FROM gadget_types WHERE type_id = 1 Error Code: 1451. Cannot delete or update a parent row: a foreign key constraint fails ('bd_tema8', 'gad')

```
-- El error ocurre porque por defecto está activa la
referencia RESTRICT.
```

8. Restricciones

CLAVE EXTERNA (AJENA, FORÁNEA O SECUNDARIA)

Siguen los ejemplos:

-- Para borrar una fila de la tabla "gadget_types", es necesario eliminar primero todas las filas referenciadas de la tabla "gadgets".

-- Por lo tanto, primero se elimina la clave foránea de la tabla "gadgets".

```
ALTER TABLE gadgets DROP CONSTRAINT fk_type;
```

-- Y posteriormente se añade nuevamente con las opciones ON DELETE SET NULL y ON UPDATE SET NULL para que cuando haya una modificación en la tabla hija, se pongan nulos en la tabla relacionada.

```
ALTER TABLE gadgets ADD CONSTRAINT fk_type FOREIGN KEY
(type_id) REFERENCES gadget_types(type_id)
ON DELETE SET NULL
ON UPDATE SET NULL;
```

-- Se intenta borrar una fila de la tabla "gadget_types"

```
DELETE FROM gadget_types WHERE type_id = 1;
```

-- Ahora sin problemas, vemos el resultado:

gadget_id	gadget_name	type_id
1	Amazon Kindle	NULL
2	Apple iPod	NULL
3	Audio Highway Listen Up	NULL
4	Apple iPad	2
5	MicroSD	2
6	Apple iPhone	3
7	BlackBerry 6210	3
8	Pager	3
9	Air Taser Model 34000	4
10	Credit Card	4
11	Zippo	4

-- Se intenta actualizar una fila de la tabla "gadget_types"

```
UPDATE gadget_types SET type_id = 20 WHERE type_id = 2;
```

-- En este caso, también se ponen en la tabla base las filas relacionadas como nulos, tal y como se estableció en la cláusula UPDATE SET NULL.

-- Ahora se puede borrar las filas "huérfanas" en la tabla base

```
DELETE FROM gadgets WHERE type_id IS NULL;
```

gadget_id	gadget_name	type_id
6	Apple iPhone	3
7	BlackBerry 6210	3
8	Pager	3
9	Air Taser Model 34000	4
10	Credit Card	4
11	Zippo	4
12	Casio G-Shock DW-5000C	4
13	Nikon F	5
14	Canon EOS 5D Mark II	5

-- Ahora queremos probar el funcionamiento de la opción CASCADE

-- Para ello, primero eliminamos la FOREIGN KEY anteriormente definida

```
ALTER TABLE gadgets DROP CONSTRAINT fk_type;
```

-- Y la volvemos a definir con las opciones CASCADE oportunas

```
ALTER TABLE gadgets ADD CONSTRAINT fk_type
FOREIGN KEY(type_id) REFERENCES
gadget_types(type_id)
ON DELETE CASCADE
ON UPDATE CASCADE;
```

-- Probamos a borrar en la tabla "gadget_types" aquel con id=3

```
DELETE FROM gadget_types WHERE type_id = 3;
```

-- Si vemos qué ha pasado en la tabla base, podemos comprobar que se han eliminado también las opciones relacionadas con el borrado anterior en la tabla sobre la que se aplicó la consulta.

-- Ahora se prueba a actualizar el "gadget_type" id de 4 a 40.

```
UPDATE gadget_types SET type_id = 40 WHERE
type_id = 4;
```

-- Y se comprueba que los cambios también se llevan a cabo en la tabla base relacionada

***HACER EJERCICIO 8**

8. Restricciones

RESTRICCIÓN DE VALIDACIÓN

Son **restricciones que dictan una condición que deben cumplir los contenidos de una columna**. Hay que utilizarla con precaución ya que puede ser origen de problemas significativos de rendimiento. Antes de que una fila sea insertada o actualizada, se evalúan todas las restricciones en el orden en el que son definidas. Si alguna restricción devuelve *falso*, la fila no será insertada o actualizada.

Antes de MariaDB 10.2.1 las expresiones de este tipo, aunque aceptadas a nivel sintáctico, eran ignoradas en este SGBD. A partir de MariaDB 10.2.1, se introdujo un nuevo método de introducir restricciones (*CONSTRAINT*):

- **CHECK** (*expression*).
- **CONSTRAINT** [*nombre_restricción*] **CHECK** (*expression*).

Expression es una condición que debe ser de tipo booleano. Si esta expresión se evalúa como *TRUE* ó *NULL*, se aceptan los valores; si la *expression* se evalúa como *FALSE*, se viola la restricción y se rechazan los valores.

La expresión solo puede usar columnas de la tabla (excepto columnas **AUTO_INCREMENT**), valores literales y funciones “deterministas” (devuelven el mismo valor en cada llamada); por ejemplo, funciones como **RAND()** ó **NOW()** **no están permitidas**. Las funciones o variables almacenadas no están permitidas. Una columna implicada en una restricción de clave foránea con una acción **ON DELETE SET NULL** u **ON UPDATE SET NULL** tampoco se puede utilizar en una restricción **CHECK**.

Una restricción **CHECK** que consta de una sola columna se puede definir directamente en las opciones de columna. De una manera más general, la sentencia **CONSTRAINT** se puede utilizar después de la lista de las columnas.

Como ya se han indicado anteriormente, se pueden deshabilitar todas las expresiones de restricción **CHECK** estableciendo la variable *check_constraint_checks* a *OFF*. Esto es útil, por ejemplo, cuando se quiere cargar una tabla que violaría algunas restricciones que posteriormente se quieran fijar.

Una misma columna puede tener múltiples **CHECKs** en su definición (en este caso, se pondrían varios **CONSTRAINT** seguidos).

8. Restricciones

RESTRICCIÓN DE VALIDACIÓN

Ejemplos:

1) Utilizando la sintaxis CHECK (expression)

```
CREATE TABLE classes(
  class_id INT AUTO_INCREMENT,
  class_name VARCHAR(255) NOT NULL,
  student_count INT CHECK(student_count > 0),
  PRIMARY KEY (class_id)
);
```

2) Utilizando la sintaxis CONSTRAINT [nombre_restricción] CHECK (expression).

```
CREATE TABLE classes(
  class_id INT AUTO_INCREMENT,
  class_name VARCHAR(255) NOT NULL,
  student_count INT,
  CONSTRAINT positive_student_count
  CHECK (student_count > 0),
  PRIMARY KEY(class_id)
);
```

-- Si se intenta insertar una fila que haga que salte la restricción...

```
INSERT INTO classes(class_name, student_count)
VALUES('MariaDB for Developers', 0);
```

28 12:18:45 insert into classes(class_name, student_count) values('... Error Code: 4025. CONSTRAINT 'classes.student_count' fa

3) Crear varias restricciones que hagan referencia a múltiples columnas

```
CREATE TABLE classes(
  class_id INT AUTO_INCREMENT,
  class_name VARCHAR(100) NOT NULL,
  begin_date DATE NOT NULL,
  end_date DATE NOT NULL,
  student_count INT,
  CONSTRAINT positive_student_count
  CHECK(student_count > 0),
  CONSTRAINT valid_date
  CHECK (end_date >= begin_date),
  PRIMARY KEY(class_id)
);
```

-- Si se intenta insertar...

```
INSERT INTO classes(class_name, begin_date, end_date,
student_count) values('MariaDB for
Developers', '2021/02/10', '2021/01/20', 10);
```

48 12:30:10 insert into classes(class_name, begin_date, end_d... Error Code: 4025. CONSTRAINT 'valid_date' failed for 'bd_tema8' 'classes'

-- Ya que las fechas no cumplen la restricción oportuna

4) Crear varias restricciones sobre una misma columna

```
CREATE TABLE classes(
  class_id INT AUTO_INCREMENT,
  class_name VARCHAR(100) NOT NULL,
  begin_date DATE NOT NULL,
  end_date DATE NOT NULL,
  student_count INT,
  CONSTRAINT positive_student_count
  CHECK(student_count > 0),
  CONSTRAINT valid_date
  CHECK (end_date >= begin_date),
  CONSTRAINT valid_begin_date
  CHECK (begin_date >= '2020/01/01'),
  PRIMARY KEY(class_id)
);
```

```
CREATE TABLE classes(
```

-- Si se intenta insertar...

```
INSERT INTO classes(class_name, begin_date,
end_date, student_count) values('MariaDB for
Developers', '2010/01/20', '2021/02/10', 10);
```

52 12:35:44 insert into classes(class_name, begin_date, end_date, s... Error Code: 4025. CONSTRAINT 'valid_begin_date' failed for 'bd_tema8' 'classes'

-- Ya que la fecha inicial no cumple una de las restricciones dadas

Se pueden añadir o borrar restricciones con la sentencia **ALTER TABLE...** oportuna:

```
ALTER TABLE classes
ADD CONSTRAINT valid_begin_date
CHECK(begin_date >= '2019-01-01');
```

```
ALTER TABLE classes
DROP CONSTRAINT valid_begin_date;
```

***HACER EJERCICIOS 9 Y 10**

8. Restricciones

MOSTRAR RESTRICCIONES

Para mostrar las **restricciones de tipo CHECK existentes**, se puede utilizar la siguiente sentencia (en versiones posteriores a MariaDB 10.2.22):

```
SELECT constraint_schema as table_schema,
       table_name,
       constraint_name,
       check_clause AS definition
FROM information_schema.check_constraints
ORDER BY table_schema, table_name;
```

Donde:

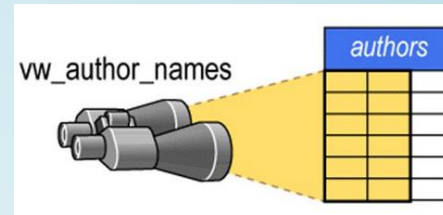
- **Table_schema** – Base de datos (esquema) donde las restricciones se definen.
- **Table_name** – Tabla donde las restricciones se definen.
- **Constraint_name** – Nombre de las restricciones.
- **Definition** – Definición de la constraint CHECK.

Ejemplo:

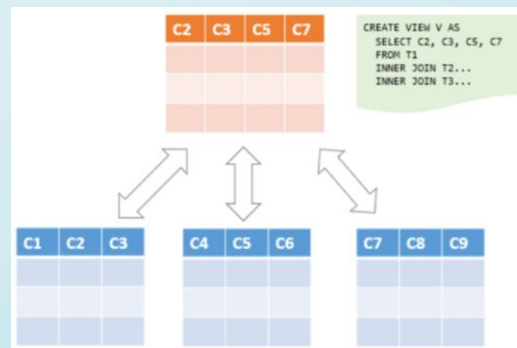
table_schema	table_name	constraint_name	definition
bd_tema8	classes	positive_student_count	`student_count` > 0
bd_tema8	classes	valid_date	`end_date` >= `begin_date`
bd_tema8	classes	valid_begin_date	`begin_date` >= '2020/01/01'
mysql	global_priv	Priv	json_valid('Priv')
test	cuentas	saldo	`saldo` >= 0

9. Vistas

Una **vista** es una tabla lógica basada en otra tabla o vista. Dicho de otra manera, es una **ventana a través de la cual se pueden consultar datos y, a veces, modificarlos**. Permiten crear **tablas virtuales** específicas para un dominio o para un tipo de usuarios. La vista no almacena datos; los datos presentados por la vista vienen de las tablas examinadas por la consulta de la vista.



Ejemplo gráfico de vista sobre determinadas filas de una tabla



Ejemplo gráfico de vista sobre determinadas filas de varias tablas

Las vistas **sirven para poder almacenar una consulta**, es decir, **para poder repetir una consulta sin tener que volver a escribirla**. Cada vez que se invoca a una vista, vuelve a consultar los datos para sacarlos actualizados. Esta es, si cabe, la principal ventaja de las vistas, su continua y automática actualización tras la modificación de alguna de las tablas a las que hace referencia.

La vista no se almacena (de manera general) físicamente sobre el disco (algunos SGBD sí ofrecen almacenarlas en disco), sino que se carga en memoria en su primera actualización. La recuperación de los datos de una vista es muy rápida a partir de la segunda ejecución. En términos de rendimiento, **las vistas son una solución a tener en cuenta siempre que se utilice varias veces el mismo *SELECT* en una sesión**.

9. Vistas

La consulta base de una vista puede ser muy sencilla o muy compleja, como se desee. **Si se suprime una tabla cuyos datos se usan en vistas, las vistas asociadas a ella se invalidan.**

No contiene datos en sí misma, sino que su información está basada en la de otras tablas, llamadas tablas base. Por lo tanto, **no podremos actualizar datos sobre una vista, habría que hacerlo sobre las tablas a las que hace referencia.**

En resumen, las vistas ofrecen las siguientes ventajas:

- **Control de accesos de una tabla real:** se puede escoger qué información específicamente se desea compartir con otros usuarios. De este modo, ellos no tendrán acceso al resto de los datos de la tabla, solo a las vistas.
- **Simplicidad y mejora del rendimiento:** se pueden crear consultas a partir de vistas que han sido extraídas de *SELECT* complejas. Esto evita tener que realizar y ejecutar esas consultas complejas una y otra vez.
- **Pruebas seguras:** las vistas ofrecen un entorno de tablas de prueba para que los desarrolladores no afecten la información real.
- **Reusabilidad de consultas:** gracias a las vistas, no se deben crear consultas complejas que requieran uniones de manera repetida.
- **Mantenimiento de la integridad:** al crear aplicaciones y usar las vistas en vez de las tablas reales, se garantiza que dichas aplicaciones no se rompan cuando se realicen cambios en la estructura de la base de datos.
- **Facilita el manejo de grandes volúmenes de información,** haciéndolos más fáciles y rápidos de manipular.

La sintaxis básica para crear una vista es la siguiente:

```
CREATE
  [OR REPLACE]
  VIEW [IF NOT EXISTS] view_name [(column_list)]
  AS select_statement
```

Para crear vistas se requiere tener el privilegio **CREATE VIEW**. Si además se utiliza la opción **OR REPLACE**, se necesitará también el privilegio **DROP** para la vista.

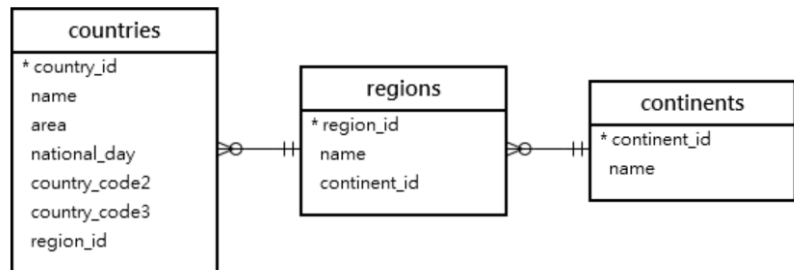
Si se especifica la opción **OR REPLACE**, si la vista ya existe, entonces la reemplaza por la nueva definición especificada. Por lo tanto, si la vista ya existe, **CREATE OR REPLACE VIEW** sería igual que utilizar la sentencia **ALTER VIEW**. De igual forma, se puede especificar la cláusula **IF NOT EXISTS** para crear la vista sólo si ésta no existe. Lógicamente, **IF NOT EXISTS y OR REPLACE no se pueden utilizar al mismo tiempo.**

9. Vistas

Las tablas y las vistas comparten el mismo espacio de nombres en la base de datos, por lo que **tablas y vistas no pueden tener el mismo nombre**.

Ejemplos:

Se supone que se dispone de las siguientes tablas:



```
CREATE TABLE continents(
  continent_id INT
  AUTO_INCREMENT,
  name VARCHAR(255) NOT
  NULL,
  PRIMARY KEY(continent_id)
);
```

```
CREATE TABLE regions(
  region_id INT AUTO_INCREMENT, name VARCHAR(100) NOT NULL, continent_id INT
  NOT NULL, PRIMARY KEY (region_id), FOREIGN KEY(continent_id) REFERENCES
  continents(continent_id) );
```

-- Imaginemos que **deseamos obtener información detallada de los países, incluyendo a las regiones y los continentes.**

Para ello, podríamos utilizar una sentencia como la siguiente.

```
SELECT c.name country, r.name región, t.name continent, area
```

```
FROM countries INNER JOIN regions r USING (region_id) INNER JOIN continents t USING (continent_id)
```

```
ORDER BY country;
```

```
CREATE TABLE countries (
  country_id INT AUTO_INCREMENT,
  name VARCHAR(50) NOT NULL,
  area DECIMAL(10,2) NOT NULL,
  national_day DATE,
  country_code2 CHAR(2) NOT NULL
  UNIQUE,
  country_code3 CHAR(3) NOT NULL
  UNIQUE,
  region_id INT NOT NULL,
  FOREIGN KEY(region_id)
  REFERENCES
  regions(region_id),
  PRIMARY KEY(country_id));
```

9. Vistas

Siguen los ejemplos de la diapositiva anterior:

-- Para **reutilizar esta consulta**, habría que guardarla en un fichero y, cada vez que la queramos utilizar, abrir dicho fichero o copiar-pegar el contenido de la consulta y ejecutarla.

-- En lugar de eso, se podría **realizar una vista**, que es un objeto que se almacena directamente en el diccionario de datos asociado a la base de datos.

```
CREATE VIEW country_details AS
  SELECT c.name country, r.name región, t.name continent, area
FROM countries INNER JOIN regions r USING (region_id) INNER JOIN continents t USING (continent_id)
  ORDER BY country;
```

-- Una vez creada esta vista, podemos utilizarla en una sentencia *SELECT* como si fuera una tabla.

```
SELECT * FROM country_details;
```

Para ver información de las vistas creadas se pueden utilizar los comandos **SHOW**, por un lado, y **DESCRIBE** ó **EXPLAIN**, por otro.

Ejemplo:

-- Para **mostrar información de una vista en concreto...**

```
SHOW CREATE VIEW country_details;
```

View	Create View	character_set_client	collation_connection
country_details	CREATE ALGORITHM=UNDEFINED DEFINER='root'@'localhost' SQL SECURITY DEFINER VIEW `country_details` AS select '...'	utf8mb4	utf8mb4_general_ci

-- Para **mostrar todas las tablas y vistas de la base de datos actual...**

```
SHOW FULL TABLES;
```

Tables_in_bd_tema8	Table_type
alumnos	BASE TABLE
articulos	BASE TABLE
continents	BASE TABLE
countries	BASE TABLE
country_details	VIEW

9. Vistas

Siguen los ejemplos de la diapositiva anterior:

`DESCRIBE country_details;` ó `EXPLAIN country_details;`

Field	Type	Null	Key	Default	Extra
country	varchar(50)	NO		HULL	
region	varchar(100)	NO		HULL	
continent	varchar(255)	NO		HULL	
area	decimal(10,2)	NO		HULL	

También es posible crear vistas basadas en otras vistas:

```
CREATE VIEW continent_areas AS
SELECT continent, SUM(area) total_area
FROM country_info – Se trata de una vista
GROUP BY continent;
```

Incluso se pueden crear vistas complejas, basadas en agrupamientos, combinaciones de tablas....

```
CREATE VIEW areas
AS SELECT
continents.name continent,
regions.name region,
SUM(area) total_area
FROM countries
INNER JOIN regions
USING (region_id)
INNER JOIN continents
USING (continent_id)
GROUP BY region, continent;
```

9. Vistas

Vistas utilizadas para inserciones y actualizaciones de datos

Las vistas también **pueden utilizarse para insertar o actualizar datos de la tabla base de la vista, pero con ciertas limitaciones**; entre otras:

- No pueden afectar a más de una tabla consultada.
- No se pueden cambiar los campos resultado de un cálculo.
- La vista no puede tener funciones de agrupamiento (count - max - min - sum - avg).
- No puede tener la cláusula distinct, union, group by, having.
- No puede tener subconsultas en la cláusula select.

Ejemplo:

-- Supongamos que se parte de la siguiente situación de tablas...

DROP TABLE IF EXISTS alumnos;

DROP TABLE IF EXISTS profesores;

```
CREATE TABLE alumnos(
  documento CHAR(8),
  nombre VARCHAR(30),
  nota DECIMAL(4,2),
  codigoprofesor INT,
  PRIMARY KEY(documento)
);
```

```
CREATE TABLE profesores (
  codigo INT
  AUTO_INCREMENT,
  nombre VARCHAR(30),
  PRIMARY KEY(codigo)
);
```

```
INSERT INTO alumnos VALUES('30111111','Ana Algarbe', 5.1, 1);
INSERT INTO alumnos VALUES('30222222','Bernardo Bustamante', 3.2, 1);
INSERT INTO alumnos VALUES('30333333','Carolina Conte',4.5, 1);
INSERT INTO alumnos VALUES('30444444','Diana Dominguez',9.7, 1);
INSERT INTO alumnos VALUES('30555555','Fabian Fuentes',8.5, 2);
INSERT INTO alumnos VALUES('30666666','Gaston Gonzalez',9.70, 2);
```

```
INSERT INTO profesores(nombre) VALUES ('Maria Luque');
INSERT INTO profesores(nombre) VALUES ('Jorge Dante');
```

DROP VIEW IF EXISTS vista_nota_alumnos_aprobados;

-- Creamos una vista con los datos de todos los alumnos que tienen

-- una nota mayor o igual a 7, junto con el nombre del profesor que

-- lo calificó

```
CREATE VIEW vista_nota_alumnos_aprobados AS
SELECT documento, a.nombre AS nombrealumno, p.nombre AS nombreprofesor, nota, codigoprofesor
FROM alumnos AS a JOIN profesores AS p ON a.codigoprofesor=p.codigo
WHERE nota>=7;
```

9. Vistas

Sigue ejemplo de la diapositiva anterior:

*SELECT * FROM vista_nota_alumnos_aprobados;*

documento	nombrealumno	nombreprofesor	nota	codigoprofesor
30444444	Diana Dominguez	Maria Luque	9.70	1
30555555	Fabian Fuentes	Jorge Dante	8.50	2
30666666	Gaston Gonzalez	Jorge Dante	9.70	2

-- Mediante la vista insertamos un nuevo alumno calificado por el profesor

-- con código 1 (los datos se insertan realmente en la tabla base)

INSERT INTO vista_nota_alumnos_aprobados

(documento, nombrealumno, nota, codigoprofesor) VALUES('99999999','Rodriguez Pablo', 10, 1);

*SELECT * FROM vista_nota_alumnos_aprobados;*

documento	nombrealumno	nombreprofesor	nota	codigoprofesor
30444444	Diana Dominguez	Maria Luque	9.70	1
30555555	Fabian Fuentes	Jorge Dante	8.50	2
30666666	Gaston Gonzalez	Jorge Dante	9.70	2
99999999	Rodriguez Pablo	Maria Luque	10.00	1

-- si consultamos la tabla base: alumnos tenemos una nueva fila con el alumno

-- insertado

*SELECT * FROM alumnos;*

documento	nombre	nota	codigoprofesor
30111111	Ana Algarbe	5.10	1
30222222	Bernardo Bustamante	3.20	1
30333333	Carolina Conte	4.50	1
30444444	Diana Dominguez	9.70	1
30555555	Fabian Fuentes	8.50	2
30666666	Gaston Gonzalez	9.70	2

-- modificamos la nota de un alumno aprobado mediante la vista

UPDATE vista_nota_alumnos_aprobados SET nota=10

WHERE documento='30444444';

*SELECT * FROM alumnos;*

documento	nombre	nota	codigoprofesor
30111111	Ana Algarbe	5.10	1
30222222	Bernardo Bustamante	3.20	1
30333333	Carolina Conte	4.50	1
30444444	Diana Dominguez	10.00	1
30555555	Fabian Fuentes	8.50	2
30666666	Gaston Gonzalez	9.70	2
99999999	Rodriguez Pablo	10.00	1

INSERT vista_nota_alumnos_aprobados(documento, nombrealumno, nota, codigoprofesor) VALUES('88888888','Laura Robles', 3, 1);

-- ... Realmente comprobamos que el dato se cargó en la tabla 'alumnos'...

*SELECT * FROM alumnos;*

documento	nombre	nota	codigoprofesor
30111111	Ana Algarbe	5.10	1
30222222	Bernardo Bustamante	3.20	1
30333333	Carolina Conte	4.50	1
30444444	Diana Dominguez	10.00	1
30555555	Fabian Fuentes	8.50	2
30666666	Gaston Gonzalez	9.70	2
88888888	Laura Robles	3.00	1
99999999	Rodriguez Pablo	10.00	1

*-- pero, lógicamente, **no se visualiza en la vista***

*SELECT * FROM vista_nota_alumnos_aprobados;*

9. Vistas

Para evitar el tipo de “inconsistencia” de la diapositiva anterior existe la cláusula **WITH CHECK OPTION**. Si se agrega dicha cláusula cuando se crea la vista, no se permitirán inserciones o actualizaciones cuando los cambios no se vayan a visualizar en la vista, es decir, **esta cláusula puede utilizarse en una vista actualizable para evitar inserciones o actualizaciones excepto en los registros en que la cláusula WHERE de la sentencia_select se evalúe como TRUE.**

Ejemplo:

-- Sobre un caso similar al anterior, utilizaremos la opción WITH CHECK OPTION y veremos qué pasa...

DROP TABLE IF EXISTS alumnos;

DROP TABLE IF EXISTS profesores;

```
CREATE TABLE alumnos(
  documento CHAR(8),
  nombre VARCHAR(30),
  nota DECIMAL(4,2),
  codigoprofesor INT,
  PRIMARY KEY(documento)
);

CREATE TABLE profesores (
  codigo INT
  AUTO_INCREMENT,
  nombre VARCHAR(30),
  PRIMARY KEY(codigo)
);
```

```
INSERT INTO alumnos VALUES('30111111','Ana Algarbe', 5.1, 1);
INSERT INTO alumnos VALUES('30222222','Bernardo Bustamante', 3.2,
1);
INSERT INTO alumnos VALUES('30333333','Carolina Conte',4.5, 1);
INSERT INTO alumnos VALUES('30444444','Diana Dominguez',9.7, 1);
INSERT INTO alumnos VALUES('30555555','Fabian Fuentes',8.5, 2);
INSERT INTO alumnos VALUES('30666666','Gaston Gonzalez',9.70, 2);

INSERT INTO profesores(nombre) VALUES ('Maria Luque');
INSERT INTO profesores(nombre) VALUES ('Jorge Dante');
```

DROP VIEW IF EXISTS vista_nota_alumnos_aprobados;

-- Se ha quitado el JOIN utilizado en el ejemplo anterior, porque la cláusula WITH CHECK OPTION presenta problemas cuando se trabaja con más vistas de más de una tabla...

CREATE VIEW vista_nota_alumnos_aprobados AS

SELECT documento, a.nombre AS nombrealumno, nota, codigoprofesor FROM alumnos AS a WHERE nota>=7 **WITH CHECK OPTION**;

-- Se genera error ya que este alumno no aparecerá en la vista (se intenta actualizar la nota de un alumno a 1, cuando, según la vista, ésta tiene que ser >=7)

UPDATE vista_nota_alumnos_aprobados SET nota=1 WHERE documento='30444444';

40 20:49:47 UPDATE vista_nota_alumnos_aprobados SET nota=1 WHERE documento='30444444' Error Code: 1369, CHECK OPTION failed 'bd_tema8`.`vista_nota_alumnos_aprobados'

9. Vistas

Sigue ejemplo anterior:

-- Se genera un error ya que se intenta utilizar la vista para insertar un alumno en la tabla base con nota de 3, cuando la vista indica que tiene que ser ≥ 7

```
INSERT vista_nota_alumnos_aprobados(documento, nombrealumno, nota, codigoprofesor) VALUES('73777777','Raquel Montes', 3, 1);
```

41 20:51:50 INSERT vista_nota_alumnos_aprobados(documento, nombrealumno, nota, codigoprofesor) ... Error Code: 1369. CHECK OPTION failed 'bd_tema8.vista_nota_alumnos_aprobados'

-- Se efectúa la inserción en forma correcta

```
INSERT vista_nota_alumnos_aprobados(documento, nombrealumno, nota, codigoprofesor) VALUES('73777777','Raquel Montes', 7, 1);
```

```
SELECT * FROM vista_nota_alumnos_aprobados;
```

documento	nombrealumno	nota	codigoprofesor
30444444	Diana Dominguez	9.70	1
30555555	Fabian Fuentes	8.50	2
30666666	Gaston Gonzalez	9.70	2
73777777	Raquel Montes	7.00	1

Una explicación más detallada se puede consultar en la documentación oficial de MariaDB: <https://mariadb.com/kb/en/inserting-and-updating-with-views/>.

Para **eliminar vistas** se puede utilizar el comando **DROP VIEW...**; permite eliminar definitivamente una vista y su contenido. Se trata de un comando definitivo, es decir, es imposible recuperar la vista si se ha eliminado por error. Este comando no tiene ninguna influencia en las tablas que se utilizan en el SELECT de creación de la vista.

Ejemplo:

```
DROP VIEW areas;
```

***HACER EJERCICIOS DEL 11 AL 17**

10. Secuencias

Una secuencia es un **objeto que genera números enteros únicos de forma automática**. Es una **alternativa a *AUTO_INCREMENT*** cuando se desea más control sobre cómo los números son generados en la secuencia y además, al poderse almacenar en caché, puede ser mucho más rápido que usar *AUTO_INCREMENT*. Otro beneficio es que se puede acceder al último valor generado por todas las secuencias utilizadas. La **sintaxis** para crear una secuencia es:

```
CREATE [OR REPLACE] [TEMPORARY] SEQUENCE [IF NOT EXISTS] sequence_name
[ INCREMENT [ BY | = ] increment ]
[ MINVALUE [=] minvalue | NO MINVALUE | NOMINVALUE ]
[ MAXVALUE [=] maxvalue | NO MAXVALUE | NOMAXVALUE ]
[ START [ WITH | = ] start ]
[ CACHE [=] cache | NOCACHE ] [ CYCLE | NOCYCLE ]
[table_options]
```

Los parámetros (no influye su orden) que puede tomar son:

- **INCREMENT**, especifica el incremento a usar. Puede ser negativo. Por defecto es 1. Si se especifica 0, la secuencia utilizará el valor de la variable del sistema *auto_increment_increment*.
- **MINVALUE**, especifica un valor mínimo para la secuencia.
 - Si *INCREMENT*>0, el valor por defecto de este parámetro es 1.
 - Si *INCREMENT*<0, el valor por defecto de este parámetro es -9223372036854775807.
- **MAXVALUE** especifica un valor máximo para la secuencia.
 - Si *INCREMENT*>0, el valor por defecto de este parámetro es 9223372036854775806.
 - Si *INCREMENT*<0, el valor por defecto de este parámetro es -1.
- **START**, Primer valor que la secuencia generará.
 - Si *INCREMENT*>0, el valor por defecto de este parámetro es *MINVALUE*.
 - Si *INCREMENT*<0, el valor por defecto de este parámetro es *MAXVALUE*.
- **CACHE**, especifica cuántos valores serán preasignados y mantenidos en la memoria del servidor. Por defecto es 1000.
- **CYCLE**, se utiliza cuando la secuencia debe comenzar de nuevo con *MINVALUE* después de que haya llegado al último de sus valores. Por defecto, el valor de este parámetro es *NOCYCLE*.

10. Secuencias

CREATE SEQUENCE requiere tener el privilegio **CREATE**.

Ejemplos:

-- Puede parecer error al escribir estas sentencias, pero se ejecutan correctamente

```
CREATE SEQUENCE s START WITH 100 INCREMENT BY 10;
```

```
CREATE SEQUENCE s2 START WITH -100 INCREMENT BY -10;
```

```
CREATE SEQUENCE s3 START WITH -100 INCREMENT BY 10 MINVALUE=-100 MAXVALUE=1000;
```

La orden **SHOW CREATE SEQUENCE nombre_secuencia;** muestra la sentencia de creación de la secuencia dada. La orden **SHOW TABLES** es capaz de listar las tablas, vistas y secuencias de la base de datos dada.

Ejemplos:

```
CREATE SEQUENCE s1 START WITH 50;
```

```
SHOW CREATE SEQUENCE s1;
```

Table	Create Table
s1	CREATE SEQUENCE `s1` start with 50 minvalue 1 maxvalue 9223372036854775806 increment by 1 cache 1000 nocycle ENGINE=InnoDB

```
SHOW TABLES;
```

Tables_in_bd_tema8
nombres
pedidos
profesores
s
s2
s3
...

-- s, s2 y s3 son secuencias; vista_nota_alumnos_aprobados es una vista; el resto son tablas.

10. Secuencias

Se puede **modificar** una secuencia con la orden **ALTER SEQUENCE** oportuna. Por ejemplo, se podría reiniciar la secuencia otro valor. La sintaxis es igual a la de *CREATE SEQUENCE*.

Ejemplos:

```
ALTER SEQUENCE s1 MINVALUE 10 START 10 RESTART 10;
```

```
ALTER SEQUENCE s RESTART 50;
```

Para **borrar** una secuencia se utiliza la sentencia **DROP SEQUENCE**. Se debe tener el privilegio *DROP* para poder aplicar esta orden. Esta orden sólo elimina secuencias, sin embargo, *DROP TABLE* puede eliminar tanto secuencias como tablas.

Ejemplo:

```
DROP SEQUENCE s1;
```

Utilizando objetos de tipo secuencia

Para obtener el **próximo valor de una secuencia**, se puede utilizar:

- **NEXT VALUE FOR** nombre_secuencia.
- **NEXTVAL**(nombre_secuencia)

Para obtener **el último valor utilizado** por la actual conexión en el uso de una secuencia, se emplea:

- **PREVIOUS VALUE FOR** nombre_secuencia.
- **LASTVAL** (nombre_secuencia)

Si la secuencia no ha sido usada todavía por la conexión actual, la llamada a esta función devuelve *NULL*.

Ejemplos:

```
SELECT LASTVAL(s);
```

```
SELECT NEXTVAL(s);
```

A partir de la versión 10.3.3. de MariaDB, los valores de una secuencia se pueden utilizar también como valor por defecto de un campo. Ejemplo:

```
CREATE SEQUENCE s1;
```

```
CREATE TABLE t1 (a INT PRIMARY KEY DEFAULT (NEXT VALUE FOR s1), b INT);
```

```
INSERT INTO t1 (b) VALUES (1),(2),(10),(20);
```

10. Secuencias

Para fijar el siguiente valor a ser devuelto por una secuencia se utiliza la función **SETVAL**, cuya sintaxis es:

```
SETVAL(sequence_name, next_value, [is_used, [round]])
```

Next_value debe ser un entero.

Si el argumento *is_used* no se da o es 1 o verdadero, entonces el siguiente valor usado será uno después del valor dado. Si *is_used* es 0 o falso, el siguiente valor generado será el valor dado.

Si se usa *round*, se establecerá el valor de *round* (o el recuento del ciclo interno, comenzando en cero) para la secuencia. Si no se usa *round*, se asume que es 0. **SETVAL** no fijará un nuevo valor para la secuencia a algo menor que su valor actual. Si se quiere asignar a la secuencia un valor menor habría que utilizar **ALTER SEQUENCE**.

Ejemplo:

```
CREATE SEQUENCE s START WITH 50 INCREMENT BY 10;
```

```
SELECT NEXTVAL(s);
```

NEXTVAL(s)
50

```
SELECT SETVAL(s, 100);
```

SETVAL(s, 100)
100

```
SELECT NEXTVAL(s);
```

NEXTVAL(s)
110

```
SELECT SETVAL(s, 50);
```

SETVAL(s, 50)
NULL

```
SELECT NEXTVAL(s);
```

NEXTVAL(s)
120

Para vincular una secuencia a una tabla, se puede realizar al insertar los valores, por ejemplo:

```
-- Primero se crea la secuencia
```

```
CREATE SEQUENCE id START WITH 1 INCREMENT BY 1;
```

```
-- Se crea la tabla y se insertan valores utilizando la secuencia
```

```
CREATE TABLE demo ( id BIGINT NOT NULL, name VARCHAR (30), PRIMARY  
KEY (id));
```

```
INSERT INTO demo (id, name) VALUES (NEXT VALUE FOR id, 'alpha');
```

```
INSERT INTO demo (id, name) VALUES (NEXT VALUE FOR id, 'beta');
```

```
-- El resultado sería:
```

id	name
1	alpha
2	beta

***HACER EJERCICIOS 18 y 19**

11. Índices

Si se desea buscar un valor específico en una columna de una tabla y la columna sobre la que queremos buscar no tuviese un índice, habría que recorrer toda la tabla comparando fila a fila hasta encontrar el valor que coincide con el valor buscado. Para tablas con pocas filas puede que esto no sea un problema, pero para tablas con millones de filas esta operación sería inimaginable.

La mejor forma de optimizar el rendimiento de una consulta es creando índices sobre las columnas que se utilizan en la cláusula WHERE. Los índices **se comportan como punteros sobre las filas de la tabla y permiten determinar rápidamente cuáles son las filas que cumplen la condición de la cláusula WHERE.**

Todos los tipos de datos de MariaDB pueden ser indexados, pero **hay que tener en cuenta que no es conveniente crear un índice para cada una de las columnas de una tabla**, ya que el exceso de índices innecesarios puede provocar un incremento del espacio de almacenamiento y un aumento del tiempo para el SGBD a la hora de decidir qué índices necesita utilizar. Los índices además añaden una sobrecarga a las operaciones de **inserción, actualización y borrado**, porque **cada índice tiene que ser actualizado después de realizar cada una de estas operaciones.**

Por lo tanto, **se debe tratar de buscar un equilibrio entre el número de índices y el tiempo de respuesta de su consulta**, de modo que pueda **reducir el tiempo de respuesta de su consulta utilizando el menor número de índices posible**. Incluso hay algoritmos que se pueden consultar para intentar optimizar el uso de índices (<https://mariadb.com/kb/en/building-the-best-index-for-a-given-select/>).

11. Índices

En MariaDB hay 4 tipos de índices principales:

- **PRIMARY KEY** (UNIQUE y NOT NULL).
- **UNIQUE** (UNIQUE y puede que NULL).
- **PLAIN INDEX** (no necesariamente UNIQUE).
- **FULLTEXT INDEX** (para búsquedas de texto completo).

Los dos primeros ya se han explicado en el apartado de “restricciones”. Como añadido, se puede indicar:

- Un índice de tipo **PRIMARY KEY** no se puede crear con el comando **CREATE INDEX**. Para añadirlo después de la creación de una tabla, habría que utilizar la sentencia **ALTER TABLE**, como ya se vio en el apartado de “restricciones”.
- Un índice de tipo **UNIQUE** puede ser añadido después de que la tabla sea creada con el comando **CREATE INDEX** o bien con **ALTER TABLE**.

Ejemplos:

-- Suponiendo que se tiene la siguiente tabla...

```
CREATE TABLE Employees (
  ID TINYINT(3) UNSIGNED NOT NULL,
  First_Name VARCHAR(25) NOT NULL,
  Last_Name VARCHAR(25) NOT NULL,
  Position VARCHAR(25) NOT NULL,
  Home_Address VARCHAR(50) NOT NULL,
  Home_Phone VARCHAR(12) NOT NULL,
  Employee_Code VARCHAR(25) NOT NULL,
  PRIMARY KEY (ID));
```

-- Ejecutando DESCRIBE Employees;

Field	Type	Null	Key	Default	Extra
ID	tinyint(3) unsigned	NO	PRI		
First_Name	varchar(25)	NO			
Last_Name	varchar(25)	NO			
Position	varchar(25)	NO			
Home_Address	varchar(50)	NO			
Home_Phone	varchar(12)	NO			
Employee_Code	varchar(25)	NO			

-- Creamos índices de tipo UNIQUE

```
ALTER TABLE Employees ADD UNIQUE EmpCode
(Employee_Code);
CREATE UNIQUE INDEX HomePhone ON
Employees(Home_Phone);
```

-- Ejecutando DESCRIBE Employees;

Field	Type	Null	Key	Default	Extra
ID	tinyint(3) unsigned	NO	PRI		
First_Name	varchar(25)	NO			
Last_Name	varchar(25)	NO			
Position	varchar(25)	NO			
Home_Address	varchar(50)	NO			
Home_Phone	varchar(12)	NO	UNI		
Employee_Code	varchar(25)	NO	UNI		

Los índices de tipo **PLAIN** no es necesario que sean únicos:

```
CREATE TABLE t2 (a INT NOT NULL, b INT, INDEX
(a,b));
INSERT INTO t2 values (1,1), (2,2), (2,2);
SELECT * FROM t2;
```

a	b
1	1
2	2
2	2

-- Si se consulta la descripción de esta tabla...
DESCRIBE t2;

Field	Type	Null	Key	Default	Extra
a	int(11)	NO	MUL		
b	int(11)	YES			

-- la clave **MUL** especifica un índice que no es ni primario ni único y dicho valor se muestra en la primera columna de un índice que permite múltiples ocurrencias de un valor dado.

-- Si hubiera más valores de la columna **KEY** sobre una misma columna, dicha columna mostraría sólo uno de los valores **PRI**, **UNI** ó **MUL** en dicho orden (de más a menos prioritario).

Los índices de tipo **FULL-TEXT** admiten la indexación y la búsqueda de textos completos. Una explicación más detallada de este tipo de índices se puede consultar en <https://mariadb.com/kb/en/full-text-indexes/>

11. Índices

La **sintaxis** para la creación de índices es la siguiente:

```
CREATE [OR REPLACE] [UNIQUE|FULLTEXT|SPATIAL] INDEX
  [IF NOT EXISTS] index_name
    [index_type]
  ON tbl_name (index_col_name,...)
  [WAIT n | NOWAIT]
  [index_option]
  [algorithm_option | lock_option] ...

index_col_name:
  col_name [(length)] [ASC | DESC]

index_type:
  USING {BTREE | HASH | RTREE}

index_option:
  KEY_BLOCK_SIZE [=] value
  | index_type
  | WITH PARSER parser_name
  | COMMENT 'string'

algorithm_option:
  ALGORITHM [=] {DEFAULT|INPLACE|COPY|NOCOPY|INSTANT}

lock_option:
  LOCK [=] {DEFAULT|NONE|SHARED|EXCLUSIVE}
```

Para crear índices **es necesario el privilegio *INDEX*** para la tabla o la base de datos.

Ejemplo:

```
CREATE TABLE departamentos (
  id INT NOT NULL,
  nombre VARCHAR(50),
  localidad VARCHAR(50),
  PRIMARY KEY (id)
```

```
);
```

-- La base de datos crea un índice de forma implícita al imponer una restricción de clave primaria (o de unicidad) en una tabla; por lo tanto, la tabla anterior tendría un índice sobre el campo *id*.

-- También se pueden crear índices de forma explícita para acceder más eficientemente por alguna columna, como, por ejemplo, como se hace a continuación a través de la columna *localidad*.

```
CREATE INDEX Depto_localidad ON departamentos(localidad);
```

11. Índices

CREACIÓN Y USO DE ÍNDICES

Se puede obtener información sobre los índices creados en una tabla con la orden **SHOW INDEX FROM** *nombre_tabla*;

Ejemplo:

SHOW INDEX FROM departamentos;

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment
departamentos	0	PRIMARY	1	id	A	0	NULL	NULL		BTREE		
departamentos	1	Depto_localidad	1	localidad	A	0	NULL	NULL	YES	BTREE		

Se obtienen los siguientes campos:

- **Table:** nombre de la tabla
- **Non_unique:** 1 si el índice permite valores duplicados y 0 si debe ser único.
- **Key_name:** el nombre del índice. La clave primaria siempre se denomina 'PRIMARY'.
- **Seq_in_index:** secuencia de la columna en el índice, comenzando por 1.
- **Column_name:** nombre de la columna.
- **Collation:** A si la columna está ordenada ascendentemente en el índice o NULL si no está ordenada.
- **Cardinality:** Número estimado de valores únicos en el índice. Puede utilizarse este valor para tomar decisiones de optimización.
- **Sub_part:** NULL si la columna entera se incluye en el índice o el número de caracteres incluidos si no.
- **Packed:** NULL si el índice no está empaquetado; o cómo se empaqueta el índice.
- **Null:** para indicar si se permiten valores nulos en la columna.
- **Index_type:** cómo se almacenará el índice.
- **Comment:** otra información, como si el índice está deshabilitado.
- **Index_comment:** contenido del atributo COMMENT cuando se creó el índice.

La sentencia para **eliminar índices** es:

```
DROP INDEX [IF EXISTS] index_name ON tbl_name
[WAIT n | NOWAIT]
```

Se necesita el privilegio **INDEX** sobre la tabla o la base de datos para poder ejecutar esta sentencia.

Ejemplo:

DROP INDEX Depto_localidad ON departamentos;

11. Índices

ESTUDIO DE OPORTUNIDAD

Los índices deben crearse durante periodos de poco trabajo de la base de datos (es una operación que consume recursos). Crear un índice es muy sencillo, lo difícil es saber cuándo hacerlo.

SI es recomendable crearlo si:

- El número de inserciones, modificaciones y borrados es despreciable frente al número de consultas por esa columna.
- La tabla es muy grande.
- Hay muchas consultas por una columna que no es clave primaria.

NO es recomendable crearlo si:

- Hay muchos valores repetidos o nulos en la columna.
- La tabla es muy pequeña.
- La tabla tiene muchas inserciones, modificaciones y borrados en proporción al número de consultas por esa columna.

Es **conveniente indexar las tablas grandes por una o varias columnas**. No es necesario en tablas pequeñas, a no ser en las claves primarias o únicas. No es bueno indexar columnas que se actualizan muy frecuentemente. **Si una columna es clave primaria, ya tiene un índice asociado.**

No hay reglas establecidas para la creación de los índices. Se aconseja crear índices en columnas que se declaran como PRIMARY KEY (si el SGBD no la crea por defecto), **sobre las columnas FOREIGN KEY, sobre las columnas a las que más se acceda, las que actúen como unión entre tablas...**; pero siempre haciendo previamente un **análisis detallado de rendimiento**.

12. Optimización de consultas e índices

El comando **EXPLAIN**, además de ser sinónimo de **DESCRIBE** (descrito en el apartado 5), también **permite obtener información sobre cómo se llevarán a cabo las consultas SELECT** (desde MariaDB 10.0.5 también consultas **UPDATE** y **DELETE**). Permite **detectar cuándo se utiliza un índice, si se usa correctamente o ver si las consultas se ejecutan de forma óptima**.

Ejemplo: la tabla artículos cuenta con 32 filas. Trabajemos sobre ella para optimizar alguna de las consultas que se puedan realizar.

-- Inicialmente, la tabla artículos está definida sin índices.

EXPLAIN artículos;

Field	Type	Null	Key	Default	Extra
ARTICULO	varchar(20)	NO		NULL	
COD_FABRICANTE	smallint(6)	NO		NULL	
PESO	smallint(6)	NO		NULL	
CATEGORIA	varchar(10)	NO		NULL	
PRECIO_VENTA	decimal(6,2)	YES		NULL	
PRECIO_COSTO	decimal(6,2)	YES		NULL	
EXISTENCIAS	decimal(5,0)	YES		NULL	

-- Queremos analizar la siguiente consulta...

EXPLAIN SELECT * FROM artículos WHERE cod_fabricante=30;

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	articulos	ALL	NULL	NULL	NULL	NULL	31	Using where

-- Indica que se realiza un escaneo completo de todas las filas de la tabla (type=ALL) y que, por tanto, se recorren 31 filas (rows) para devolver el resultado

-- Como consultar por fabricantes parece que es una consulta que se puede utilizar con frecuencia, se crea un índice sobre dicha columna...

CREATE INDEX fabricante_art ON artículos(cod_fabricante);

-- Mostrando información del mismo...

SHOW INDEX FROM artículos;

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment
articulos	1	fabricante_art	1	COD_FABRICANTE	A	10	NULL	NULL		BTREE		

12. Optimización de consultas e índices

EXPLAIN artículos;

Field	Type	Null	Key	Default	Extra
ARTICULO	varchar(20)	NO		NULL	
COD_FABRICANTE	smallint(6)	NO	MUL	NULL	
PESO	smallint(6)	NO		NULL	
CATEGORIA	varchar(10)	NO		NULL	
PRECIO_VENTA	decimal(6,2)	YES		NULL	
PRECIO_COSTO	decimal(6,2)	YES		NULL	
EXISTENCIAS	decimal(5,0)	YES		NULL	

-- Si ahora se repite la consulta anterior, analizándola...

*EXPLAIN SELECT * FROM artículos WHERE cod_fabricante=30;*

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	articulos	ref	fabricante_art	fabricante_art	2	const	9	

-- Podemos comprobar que ahora la columna type=ref (indica que se ha usado un índice para encontrar las filas), necesitando recorrer sólo 9 filas, en lugar de las 31 iniciales, para devolver el resultado.

Queda fuera de este curso un estudio completo del proceso de optimización, pero se puede consultar más información en: <https://mariadb.com/kb/en/explain/> y en <https://mariadb.com/kb/en/optimization-and-tuning/>.

***HACER EJERCICIOS DEL 20 AL 28**